

Содержание

Введение.....	2
Глава 1. Анализ предметной области и проектирование системы	4
1.1 Сущность предметной области и организация работы второй линии технической поддержки.....	4
1.2 Анализ существующих программных решений	6
1.3 Формирование требований к информационной системе и обоснование её наименования.....	10
1.4 Обоснование выбора программных средств реализации.....	16
Глава 2. Разработка портала рабочих задач для 2ЛТП команд	20
2.1 Проектирование и реализация базы данных	20
2.2 Разработка пользовательских интерфейсов	29
2.3 Реализация серверной логики, системы уведомлений и аналитики	34
Заключение.....	41
Список Литературы.....	43
Приложение 1	45
Приложение 2	46
Приложение 3	47
Приложение 4	48
Приложение 5	49
Приложение 6	50
Приложение 7	51
Приложение 8	52

Введение

Команды второй линии технической поддержки (далее — 2ЛТП) разбирают нетривиальные инциденты: исследуют обстоятельства обращения, воспроизводят проблему, готовят исправление или ставят задачу разработчикам. Инженер второй линии одновременно удерживает десятки клиентских обращений, внутренних задач разной срочности и переписок в рабочих чатах. Чем крупнее команда, тем заметнее издержки, возникающие из-за отсутствия единого инструмента, который связывал бы внешнее обращение клиента с внутренней работой по его устранению.

Обычно эту потребность закрывают набором разрозненных инструментов — системами учёта обращений, трекерами задач, базами знаний и мессенджерами, между которыми возникают «слепые зоны»: обращение живёт в одной системе, задача разработке — в другой, согласование сроков — в третьей. В результате ни руководитель команды, ни менеджер не видят картину целиком, а часть обращений «теряется» при переключении между системами.

Актуальность темы дипломного проекта обусловлена тем, что для команд 2ЛТП требуется единая информационная система, покрывающая весь цикл работы — от приёма заявки клиента до закрытия связанной с ней внутренней задачи. Существующие на рынке решения либо ориентированы только на разработку, либо только на классическую поддержку, либо обладают избыточной стоимостью владения, что особенно ощутимо в компаниях среднего размера [8].

Степень разработанности темы по отдельным её составляющим высока: самостоятельно изучены системы обработки обращений, трекеры задач и веб-приложения с неблокирующей серверной частью. Вместе с тем связных решений, ориентированных именно на профиль 2ЛТП, заметно меньше, а большинство доступных материалов опирается на классические синхронные фреймворки и не учитывает специфику современного асинхронного стека. Это оставляет пространство для практической работы.

Целью дипломного проекта является разработка веб-платформы для управления рабочими задачами команд второй линии технической поддержки,

обеспечивающей сквозной процесс от приёма клиентской заявки до её закрытия через внутреннюю задачу с подзадачами и системой уведомлений.

Для достижения поставленной цели в работе решаются следующие задачи:

- 1) проанализировать особенности работы команд 2ЛТП и существующие на рынке программные решения, выделить требования к создаваемой платформе;
- 2) сформировать модель ролей с системой разграничения прав доступа и спроектировать схему реляционной базы данных, отражающую ключевые сущности предметной области и связи между ними;
- 3) обосновать выбор стека разработки, сформулировать функциональные требования к серверной и клиентской частям системы, определить требования к безопасности платформы — аутентификации, защите данных и ограничению действий;
- 4) реализовать серверную часть на FastAPI с подключением к PostgreSQL, фоновыми задачами через Celery и каналом доставки уведомлений по протоколу WebSocket;
- 5) разработать два пользовательских интерфейса на React — внутренний для сотрудников и клиентский — с учётом ролевой модели и сценариев использования;
- 6) подготовить документацию и инструкцию по развёртыванию, оформить проект в виде открытого репозитория с конфигурацией для запуска в Docker.

Объектом исследования выступает процесс организации работы команд второй линии технической поддержки.

Предметом исследования являются программно-технические средства автоматизации управления задачами и заявками в условиях работы 2ЛТП.

Глава 1. Анализ предметной области и проектирование системы

1.1 Сущность предметной области и организация работы второй линии технической поддержки

Техническая поддержка пользователей в современных IT-компаниях традиционно строится по многоуровневой модели. Первая линия (1ЛТП) принимает все входящие обращения и решает типовые вопросы по заранее подготовленным сценариям; всё, что не закрывается за несколько минут, передаётся на вторую линию — специалистам с доступом к журналам приложений, базам данных, системам мониторинга и исходному коду [8]. Основная масса сложных инцидентов обрабатывается именно на этом уровне, поэтому от организации работы 2ЛТП напрямую зависит итоговое качество поддержки продукта.

Работа 2ЛТП сочетает оперативность поддержки и глубину инженерного анализа: инженер отвечает на внешний запрос клиента, но для поиска причины действует как разработчик — воспроизводит проблему, читает код, выполняет SQL-запросы к базе. Тиражируемых инструментов, рассчитанных именно на такую гибридную работу, на рынке немного: системы учёта обращений ориентированы на общение с клиентом, а трекеры задач — на внутреннюю разработку, и совмещать их приходится вручную.

Рабочий день инженера второй линии складывается из четырёх типов активности. Первый — разбор входящих заявок: чтение обращения, уточнение деталей, классификация. Второй — воспроизведение и диагностика проблемы по журналам и данным. Третий — оформление результата: ответ клиенту, внутренняя запись или постановка задачи разработке. Четвёртый — координация с коллегами: эскалации руководителю команды и согласование сроков с менеджером. Каждый из этих типов активности порождает свои информационные потребности, которые проектируемая система должна закрывать.

Сами заявки неоднородны. Их можно условно разделить на инциденты (конкретные сбои в работе сервиса), запросы на изменение (почти всегда

превращаются в задачу разработки), консультационные вопросы и административную рутину — создание учётных записей, выдачу прав, выгрузку данных. Такое разнообразие требует, чтобы система учитывала связь внешнего обращения с порождённой им внутренней работой и позволяла проследивать эту связь в обе стороны.

Сегодня перечисленные задачи закрываются разрозненными системами: заявки ведутся в системах класса Service Desk, задачи — в трекерах, документация — в базах знаний, переписка — в мессенджерах. Интеграции между этими системами, как правило, слабые, и работа инженера превращается в постоянное переключение контекста. Именно переключение контекста является одной из главных причин «потерянных» обращений: задача, заведённая в одной системе, теряет связь с обращением в другой, и о ней забывают.

Руководитель команды (тимлид) отвечает за равномерное распределение нагрузки между инженерами и за согласованные эскалации к разработке. Без инструмента, наглядно показывающего загрузку подчинённых и статусы задач, он вынужден выполнять роль диспетчера, вручную собирающего информацию по чатам. Снять с тимлида эту рутину, дав ему наглядную картину загрузки команды и состояния задач, — одна из практических целей проекта.

Команда 2ЛТП обычно работает сразу с несколькими продуктами компании — личным кабинетом, административной панелью, мобильным приложением, аналитическим порталом, программным интерфейсом. Поэтому понятие «приложение» закладывается в модель данных как отдельный справочник: он нужен для маршрутизации обращений и для статистики по продуктам. Рядом существует понятие «сервиса» — внутреннего направления работ (фронтенд, бэкенд, эксплуатация, дизайн, инфраструктура). Разделение приложений и сервисов даёт аналитику сразу по двум независимым осям и не смешивает клиентскую сторону с внутренней.

Отдельного внимания заслуживают показатели, по которым оценивается работа второй линии: среднее время реакции и закрытия обращения, доля решённых в срок, равномерность загрузки инженеров и динамика накопления

нерешённых задач. Без единой системы эти показатели приходится собирать вручную из нескольких источников, что делает их неточными и запаздывающими. Соглашения об уровне обслуживания (SLA), фиксирующие предельные сроки реакции, при разрозненных инструментах также трудно контролировать. Следовательно, помимо ведения задач и обращений, система должна давать руководителю наглядную аналитику в реальном времени, иначе управление нагрузкой остаётся реактивным.

На вторую линию чаще попадают обращения, которые клиент уже считает затянувшимися, поэтому в клиентском интерфейсе принципиально важна ясная обратная связь по статусу: иначе клиент начинает дублировать обращение по другим каналам, увеличивая нагрузку. Таким образом, проектируемая платформа должна закрывать всю цепочку — от приёма заявки до выполнения, проверки и закрытия с информированием клиента — в рамках одной системы, с прозрачной ролевой моделью, едиными уведомлениями и статистикой по сервисам и приложениям. Эти выводы служат отправной точкой для формирования требований в последующих параграфах.

1.2 Анализ существующих программных решений

Прежде чем формулировать требования к собственной системе, целесообразно рассмотреть наиболее распространённые на рынке решения, применяемые для учёта обращений и управления задачами. Анализ проводился по таким критериям, как охват сквозного процесса «заявка → задача», стоимость владения, ориентация на профиль 2ЛТП и возможность развёртывания на собственной инфраструктуре организации.

Atlassian Jira — одна из наиболее популярных систем управления задачами в среде разработки. Она предоставляет гибкую настройку рабочих процессов, доски в стиле Scrum и Kanban, развитую систему полей, фильтров и отчётов. Однако Jira исторически ориентирована на команды разработки, а не на классическую поддержку: учёт клиентских обращений реализуется отдельным продуктом (Jira Service Management), и связка двух продуктов усложняет картину и увеличивает

стоимость владения. Для команды среднего размера полнофункциональная конфигурация оказывается избыточной и дорогой в сопровождении, а тонкая настройка требует выделенного администратора.

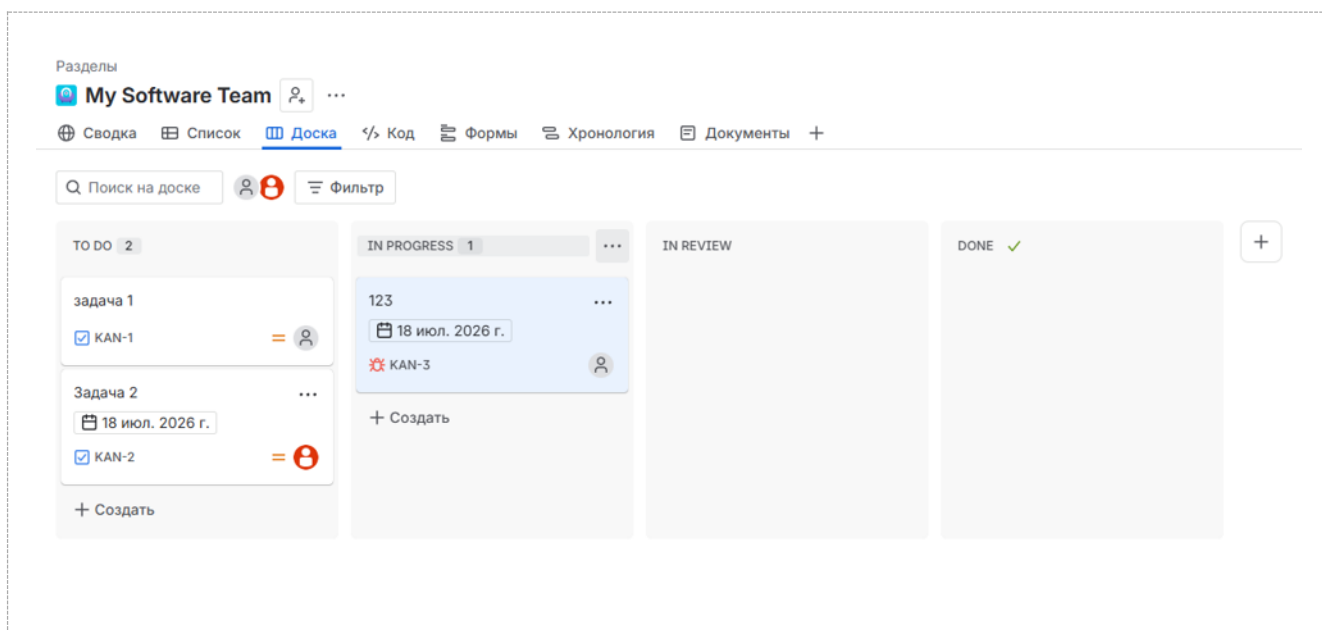


Рисунок 1 — Интерфейс доски задач в системе Atlassian Jira

Битрикс24 — российская платформа, объединяющая CRM, задачи, чаты и инструменты совместной работы. Она привлекательна широтой охвата и наличием бесплатного тарифа, однако представляет собой универсальный «комбайн»: специфика второй линии поддержки в ней не выражена, рабочие процессы задач слабо связаны с обращениями клиентов, а попытка приспособить систему под профиль 2ЛТП приводит к обилию неиспользуемых функций и перегруженному интерфейсу.

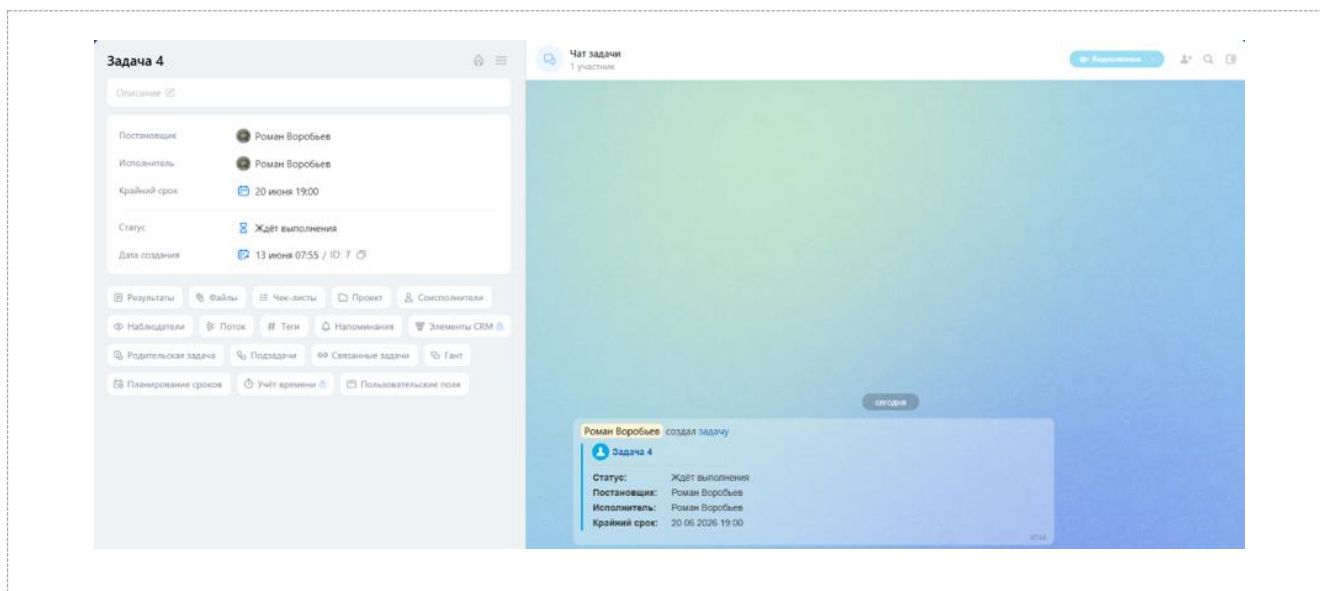


Рисунок 2 — Раздел управления задачами в Битрикс24

Яндекс Трекер — облачный трекер задач, развиваемый в составе экосистемы Яндекса. Он удобен, имеет русскоязычный интерфейс и развитую систему очередей, однако ориентирован прежде всего на внутренние задачи команд и не предоставляет отдельного клиентского портала, через который внешний заказчик мог бы подавать обращения и отслеживать их статус. Кроме того, как облачный сервис он не предполагает развёртывания на собственной инфраструктуре организации, что критично для компаний с повышенными требованиями к хранению данных.

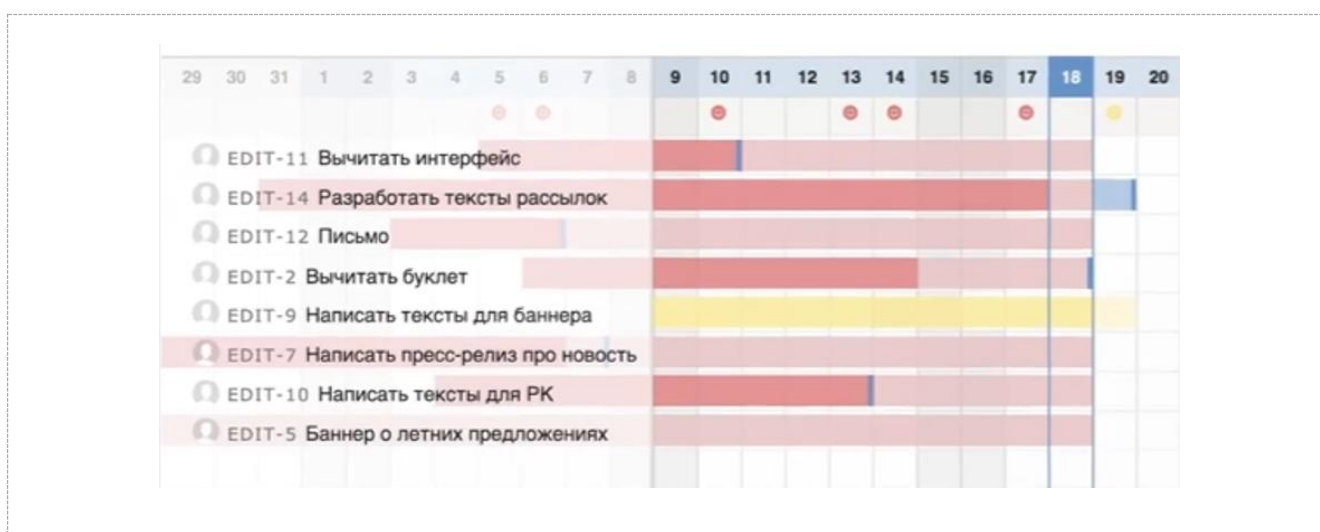


Рисунок 3 — Интерфейс очередей задач в Яндекс Трекере

Zendesk — зарубежная система класса Service Desk, ориентированная на обработку клиентских обращений. Она хорошо решает задачу учёта тикетов и общения с клиентом, располагает развитой системой автоматизаций и отчётности, но почти не покрывает внутреннюю инженерную работу: декомпозицию задачи на подзадачи, распределение их между исполнителями и согласование результата. Для команды 2ЛТП, которая совмещает поддержку и разработку, такого охвата недостаточно. Отдельно отмечается, что универсальные инструменты вроде Trello или Notion применимы для ведения списков и заметок, но не обеспечивают ни ролевой модели, ни связи обращений с задачами, ни специализированной аналитики.

Рассматривались и открытые системы, развёртываемые на собственной инфраструктуре, — Redmine, GLPI, OTRS, Zammad. Они привлекательны отсутствием лицензионных платежей и контролем над данными, однако исторически тяготеют к одному из двух полюсов: либо к классической поддержке (учёт обращений), либо к управлению проектами и задачами. Совмещение клиентской и внутренней работы в единой связной модели в них выражено слабо, интерфейсы зачастую устарели, а адаптация под профиль 2ЛТП требует значительной доработки и сопровождения. Таким образом, и среди открытых решений готового варианта, закрывающего сквозной процесс второй линии, выявить не удалось.

Сводное сравнение рассмотренных решений по ключевым критериям приведено в таблице 1.

Критерий	Jira	Битрикс24	Яндекс Трекер	Zendesk
Сквозной процесс «заявка → задача»	Частично	Слабо	Нет	Частично
Клиентский портал	Отдельный продукт	Есть	Нет	Есть
Ориентация на 2ЛТП	Нет	Нет	Нет	Нет
Развёртывание on-premise	Ограничено	Ограничено	Нет	Нет
Стоимость владения	Высокая	Средняя	Средняя	Высокая

Таблица 1 — Сравнение существующих программных решений

Анализ показывает, что ни одно из распространённых решений не покрывает сквозной процесс работы 2ЛТП целиком и одновременно: системы поддержки слабы в части внутренней разработки, трекеры задач — в части работы с внешним клиентом, а универсальные платформы избыточны и не учитывают специфику второй линии. Это обосновывает разработку специализированной платформы, спроектированной непосредственно под рассмотренную предметную область, с управляемой стоимостью владения и возможностью развёртывания на собственной инфраструктуре.

1.3 Формирование требований к информационной системе и обоснование её наименования

Опираясь на разбор предметной области и анализ аналогов, требования к платформе разделены на функциональные и нефункциональные, после чего отдельно рассмотрены ролевая модель, требования безопасности и обоснование наименования системы.

Функциональные требования к серверной части. Платформа должна предоставлять программный интерфейс (REST API), через который оба клиентских приложения работают с данными. В состав интерфейса входят следующие группы операций:

- операции с пользователями и командами — создание, обновление, деактивация, привязка к команде, смена роли;
- операции с обращениями клиентов (тикетами) — подача обращения, просмотр, изменение статуса, комментирование, отклонение с указанием причины;
- операции с задачами и подзадачами — создание, назначение команде, принятие в работу, отправка на проверку, одобрение или отклонение менеджером, архивация;
- операции со справочниками «приложения» и «сервисы» — добавление, редактирование, мягкое удаление;

- операции с комментариями к сущностям — оставление, удаление, управление видимостью для клиента;
- операции с уведомлениями — получение списка, отметка о прочтении, подсчёт непрочитанных;
- аналитика — задачи по статусам, среднее время закрытия, нагрузка команд, просроченные задачи, статистика по тикетам, недельные срезы;
- журнал действий — для администратора, с фильтрами по пользователю, типу сущности и периоду.

Функциональные требования к внутреннему интерфейсу. Состав меню переключается в зависимости от роли пользователя. У менеджера — панель показателей и разделы «Задачи», «Тикеты», «Аналитика»; у тимлида — командная панель с канбан-доской и страницей декомпозиции; у инженера — список «Мои задачи» с быстрой сменой статуса; у администратора — управление пользователями, командами и журнал аудита.

Функциональные требования к клиентскому интерфейсу. Клиент видит только свои обращения: форму создания с выбором приложения и приоритета, список с фильтром по статусам, карточку с наглядной шкалой прогресса и видимыми ему комментариями, а также уведомления и профиль. Внутренняя «кухня» работы команды клиенту недоступна.

Нефункциональные требования сформулированы исходя из условий эксплуатации в команде 2ЛТП. Платформа должна устойчиво работать при нагрузке типичной команды (20–50 сотрудников, до нескольких сотен обращений в день), обеспечивать время отклика интерфейса не более секунды и доставку уведомлений практически в реальном времени, а длительные операции (отправка писем, проверка дедлайнов) выносить в очередь фоновых задач, чтобы не замедлять ответ.

Удобство использования предполагает интерфейс без избыточных англицизмов в подписях, путь до главного действия не длиннее трёх нажатий, фильтры на таблицах и быстрый полнотекстовый поиск по задачам. Эксплуатационные требования включают развёртывание одной командой,

штатные миграции и демонстрационные данные, конфигурацию через переменные окружения и вывод журналов в стандартный поток. Расширяемость обеспечивается тем, что перечисляемые типы вынесены в отдельные перечисления, а матрица прав строится проверками вокруг ролей, поэтому новая роль или статус добавляются точечной правкой без переписывания серверной части. Поддерживаемость достигается типизацией кода и зафиксированными версиями зависимостей, что делает сборку воспроизводимой. Наконец, прозрачность действий обеспечивается журналом аудита: любая значимая операция оставляет след с указанием времени, инициатора и метаданных, что критично для разбора спорных ситуаций.

К нефункциональным требованиям отнесены также резервирование данных и наблюдаемость. Сохранность информации обеспечивается на уровне эксплуатации СУБД: постоянный том в контейнере и штатное снятие дампов позволяют делать резервные копии по расписанию, а загруженные пользователями файлы хранятся на отдельном томе, который тоже включается в стратегию резервного копирования. Журналы всех сервисов выводятся в стандартный поток, что позволяет подключить любой стандартный агрегатор журналов без изменения кода. Эти требования важны, поскольку платформа предназначена для повседневной работы команды, и потеря данных или отсутствие журналов недопустимы при разборе инцидентов.

Система ролей и разграничение прав доступа. Для описания структуры 2ЛТП выбраны пять ролей — клиент, менеджер, тимлид, инженер и администратор. Они достаточно подробно отражают организацию команды, не дробя её до уровня, на котором права становится трудно поддерживать.

Клиент — внешний заказчик, обращающийся в поддержку. Он работает только с клиентским порталом: регистрируется сам или заводится менеджером, создаёт и видит лишь свои обращения, читает только те комментарии менеджера, что помечены как видимые. Задач, подзадач, отделов и чужих обращений клиент не видит.

Менеджер отвечает за приём и сопровождение обращений и за постановку задач команде. Он видит все обращения, переводит их между статусами, добавляет

комментарии с переключателем видимости для клиента, создаёт на их основе задачи и связывает обращение с задачей, а также управляет справочниками приложений и сервисов.

Тимлид руководит отдельной командой (фронтенд, бэкенд, тестирование, эксплуатация, дизайн) и видит только её задачи. Создавать задачи с нуля он не может — это право менеджера, — но декомпозирует пришедшую задачу на подзадачи, назначает их инженерам, переводит задачу в статусы «в работе» и «на проверке». В пределах своей команды ему видны загрузка каждого инженера и канбан-доска активных задач.

Инженер (в коде роль *worker*) — рядовой работник команды. Он видит только свои подзадачи, меняет их статус, комментирует и прикрепляет файлы. Доступа к чужим подзадачам, справочникам и полной аналитике у него нет — это узкий рабочий профиль, рассчитанный на исполнение.

Администратор — техническая роль с максимальными правами: создаёт и редактирует пользователей, переназначает роли, формирует команды, читает журнал аудита и при необходимости правит данные. Это не управленческая, а скорее обслуживающая роль — один-два человека на организацию, отвечающих за работоспособность платформы.

Разграничение прав строится на двух уровнях. Первый — маршрутный, на стороне клиента: ограничение видимости пунктов меню и перенаправление на «свою» главную страницу. Второй — серверный: каждый защищённый эндпоинт проверяет право на действие и при его отсутствии возвращает ответ с кодом 403 [17]. «Защита только на стороне клиента» сознательно не допускается, поскольку пользователь способен сформировать любой HTTP-запрос вручную. Матрица прав зафиксирована заранее: создание обращения доступно только клиенту; просмотр всех обращений, создание и назначение задач, одобрение результата и управление справочниками — менеджеру и администратору; принятие задачи в работу и создание подзадач — тимлиду своей команды; смена статуса подзадачи — назначенному инженеру; управление пользователями и просмотр журнала аудита — только администратору.

Модель видимости комментариев заслуживает отдельного пояснения. По умолчанию все комментарии считаются внутренними; клиент комментирует только свои обращения и видит лишь те комментарии менеджера, что явно помечены как видимые. Гранулярных разрешений уровня «право на конкретный отдел» сознательно нет: пять крупных ролей покрывают подавляющее большинство случаев, а поддержка тонкой детализации дорога в реализации и эксплуатации. Принадлежность тимлида команде выражена двумя связями — привязкой пользователя к команде и ссылкой команды на тимлида, — и серверная логика сверяет обе: принять задачу в работу может только тимлид её команды, иначе действие отклоняется. Такой подход делает проверки прав предсказуемыми и единообразными.

Требования к безопасности формулировались по принципу «защита по умолчанию и в несколько слоёв» [20]: спорные решения разрешались в пользу более безопасного варианта, а итоговая защищённость обеспечивается наложением мер, а не единственным механизмом.

Хранение паролей. В базе хранится только криптографический хеш пароля, вычисленный алгоритмом bcrypt с двенадцатью раундами, — это разумный компромисс между скоростью авторизации и стойкостью к перебору; число раундов при необходимости увеличивается настройкой.

Аутентификация и сессии. После входа сервер выдаёт два токена JSON Web Token [19]: токен доступа на пятнадцать минут и токен обновления на семь дней, оба в куках с признаком httpOnly. Это принципиально: токен недоступен из JavaScript, что снимает значительный класс атак с его кражей. Признак SameSite уменьшает риск межсайтовой подделки запроса, а передача только по защищённому каналу в эксплуатации исключает перехват. Короткое время жизни токена доступа ограничивает окно компрометации пятнадцатью минутами [18], а токен обновления может быть отозван на сервере.

Защита от перебора и контроль доступа. Частота запросов ограничивается (не более установленного числа попыток входа и регистраций в минуту с одного адреса, с общим лимитом запросов), при превышении возвращается код 429.

Каждый защищённый маршрут декорирован зависимостью, проверяющей роль, а в обработчике дополнительно проверяется владение объектом — например, что тимлид относится к команде задачи.

Защита от инъекций и валидация ввода. Все обращения к базе идут через параметризованные запросы, а единственный фрагмент с полнотекстовым поиском передаёт параметры через привязку, а не конкатенацию строк [5]. Все входящие данные проходят через схемы валидации: некорректное значение отклоняется до обработчика с кодом 422, что одновременно повышает удобство и служит слоем защиты. Загрузка файлов ограничена по типу (проверка по содержимому и белому списку) и размеру, а имя файла очищается и сохраняется так, чтобы исключить коллизии и выход за пределы каталога.

Сетевые ограничения, журнал и привилегии. Сервер принимает запросы только с разрешённых источников — адресов внутреннего и клиентского порталов; открытие интерфейса всему интернету не допускается. Журнал аудита служит средством расследования: администратор видит, кто и когда что менял. По принципу наименьших привилегий контейнер серверной части запускается под непривилегированным пользователем, а доступ к базе и файловому хранилищу ограничен. Персональные данные клиента возвращаются интерфейсу в минимально необходимом объёме, а внутренние комментарии физически недоступны клиентскому интерфейсу за счёт фильтра на уровне запроса. В эксплуатации веб-сервер добавляет стандартные заголовки безопасности, а версии зависимостей фиксируются.

Обоснование наименования системы. Разрабатываемой платформе присвоено рабочее название «2ЛТП Task Portal» — портал рабочих задач второй линии технической поддержки. Аббревиатура «2ЛТП» прямо указывает на предметную область, для которой создаётся система, а слова «Task Portal» отражают её основную функцию: служить единой точкой входа для работы с задачами и заявками. Название одновременно краткое, отражает назначение продукта и удобно для использования в адресах и конфигурации проекта.

1.4 Обоснование выбора программных средств реализации

Выбор технологического стека выполнялся отдельно для серверной части, базы данных, очереди фоновых задач и клиентской части, исходя из сформулированных требований к производительности, безопасности и удобству сопровождения. По каждому ключевому компоненту рассматривались альтернативы, а итоговое решение обосновывалось сравнением.

Серверный фреймворк. Кандидатами на роль серверного фреймворка были Django, Flask и FastAPI. Django синхронен в своей основе — полноценная асинхронность в связке со встроенной ORM остаётся проблемной [16], — а жёсткая структура «приложений» избыточна для проекта такого размера. Flask является микрофреймворком, который пришлось бы обвешивать сторонними библиотеками для миграций, фоновых задач, WebSocket и аутентификации, постоянно следя за их совместимостью. FastAPI подошёл лучше всего [3]: его асинхронное ядро на базе Starlette, нативная поддержка конструкций `async/await`, валидация данных через Pydantic и автоматическая генерация схемы OpenAPI позволяют описать контракт один раз и одновременно получить валидацию и интерактивную документацию. Сравнение фреймворков приведено в таблице 2.

Критерий	Django	Flask	FastAPI
Асинхронность	Ограниченная	Через расширения	Нативная
Валидация данных	Формы	Вручную	Pydantic
Документация API	Через пакеты	Через пакеты	Встроенная (OpenAPI)
Готовность «из коробки»	Высокая	Низкая	Средняя
Подходит для проекта	Избыточен	Недостаточен	Оптимально

Таблица 2 — Сравнение серверных фреймворков

Для связи с базой данных выбран SQLAlchemy 2.0 в асинхронном режиме с типизацией моделей через Mapped и mapped_column, а для управления изменениями схемы — инструмент миграций Alembic [2, 5]. Эта связка зрелая, хорошо документирована и позволяет описывать модели в типизированном виде, что снижает число ошибок.

Цикл обработки запроса. Обработка входящего HTTP-запроса в выбранном стеке проходит несколько последовательных этапов. Запрос поступает на сервер Uvicorn, проходит через слой промежуточного программного обеспечения (ограничитель частоты запросов, обработчик междоменных запросов), после чего маршрутизатор FastAPI сопоставляет путь и метод соответствующему обработчику. До вызова обработчика отрабатывают зависимости — извлечение пользователя из токена и проверка прав, а входные данные проверяются схемой Pydantic. Сам обработчик обращается к базе данных через асинхронную сессию, формирует результат, который сериализуется выходной схемой и возвращается клиенту. Длительные операции при этом не выполняются синхронно, а ставятся в очередь фоновых задач, что удерживает время отклика в пределах требований.

База данных. К системе управления базами данных предъявлялись требования реляционной модели с поддержкой транзакций ACID, перечисляемых типов для статусов, полнотекстового поиска и гибкого хранения метаданных. Рассматривались PostgreSQL, MySQL и SQLite. PostgreSQL версии 15 отвечает всем требованиям: в нём есть транзакции, перечисляемые типы, тип JSONB [1], полнотекстовый поиск на основе tsvector [4] и индексы GIN. MySQL уступает по зрелости полнотекстового поиска (особенно для русского языка) и удобству работы с JSON, а SQLite не подходит для нескольких параллельных процессов, одновременно пишущих в одну базу, что требуется при работе серверной части и фоновых процессов. Сравнение систем управления базами данных приведено в таблице 3.

Критерий	PostgreSQL	MySQL	SQLite
Транзакции ACID	Полная	Полная	Ограниченная
Полнотекстовый поиск	Сильный	Слабее	Базовый
Поддержка JSON/JSONB	Зрелая	Базовая	Минимальная
Параллельная запись	Да	Да	Нет
Итоговый выбор	Выбрана	—	—

Таблица 3 — Сравнение систем управления базами данных

Очередь задач и кэш. Для фоновой обработки выбран Celery с брокером сообщений Redis [12] — классическая связка для отложенных задач. На ней держатся два процесса: рассылка почтовых уведомлений (синхронно она тормозила бы ответ интерфейса) и периодические задачи — проверка приближающихся дедлайнов и автоматическая архивация задач старше тридцати дней; расписание обеспечивает отдельный процесс `celery-beat`. Redis при этом выполняет двойную роль — брокера очереди и хранилища ограничителя частоты запросов; более тяжёлый брокер для такого масштаба избыточен.

Клиентская часть. Для разработки интерфейсов выбран React 18 с языком TypeScript [13] — у этой связки самая широкая экосистема среди библиотек пользовательского интерфейса [6], а статическая типизация снимает целый класс ошибок ещё до запуска. Сборщик — Vite, обеспечивающий быстрый запуск и режим горячей перезагрузки без сложной настройки. Управление серверным состоянием возложено на библиотеку TanStack Query [14], берущую на себя кеширование, повторную проверку данных и повторные попытки запросов, а состоянием уровня приложения управляет минималистичная библиотека Zustand. Готовые элементы интерфейса — таблицы, формы, модальные окна, выпадающие списки, всплывающие уведомления — предоставляет библиотека компонентов Ant Design 5 [15], выбранная за деловой стиль, подходящий внутреннему рабочему инструменту, и встроенную русскую локализацию.

Оформление задаётся переменными CSS на корневом уровне [10]: бренд-цвета, семантические цвета (успех, предупреждение, ошибка), нейтральные оттенки, радиусы и тени собраны в одном файле, что позволяет перенастроить тему — например, под брендинг конкретного клиента — правкой единственного места.

Организация кода. Все компоненты собраны в едином репозитории (монорепозитории): серверная часть и два фронтенда хранятся вместе, что упрощает согласованное изменение интерфейса и его потребителей и единое развёртывание. Перечисляемые типы вынесены в отдельные перечисления, а матрица прав строится проверками вокруг ролей, поэтому добавление новой роли или статуса требует точечной правки. На клиентской части применяется строгий

режим типизации без использования универсального типа, плоская структура каталогов и единое именование, а статический анализатор и форматировщик поддерживают единообразие кода.

Автоматическая генерация клиентских типов из схемы интерфейса сознательно не используется: для проекта такого размера ручное поддержание описаний типов дешевле, а компилятор подсвечивает устаревшие сигнатуры при изменении интерфейса. Тестирование на данном этапе выполняется ручным прогоном ключевых сценариев на демонстрационных данных: создать обращение клиентом, обработать менеджером, принять тимлидом, выполнить инженером, одобрить и убедиться, что связанное обращение закрылось. Полноценный конвейер автоматической сборки и проверок для дипломного проекта избыточен, но структура кода готова к его подключению.

Контейнеризация и развёртывание. Для развёртывания используются Docker и docker compose. Проект состоит из нескольких долгоживущих сервисов — PostgreSQL, Redis, серверная часть на FastAPI и Uvicorn, рабочий процесс и планировщик Celery, а также два собранных фронтенда, отдаваемых через веб-сервер nginx. Запуск всей системы сводится к одной команде, после чего остаётся применить миграции и при необходимости наполнить базу демонстрационными данными. Качество кода поддерживается простыми правилами: типизация там, где смысл не очевиден; компактные обработчики маршрутов с выносом сложной логики в сервисный слой; отсутствие «магических» строк, которые заменены перечислениями и настройками. В результате формируется единый репозиторий, содержащий серверную часть и оба клиентских приложения, связанные общей конфигурацией.

Глава 2. Разработка портала рабочих задач для 2ЛТП команд

2.1 Проектирование и реализация базы данных

Реализация началась с базы данных. Проектирование велось по последовательности «сущности предметной области → связи → атрибуты → индексы и оптимизации» [7]. На основе анализа первой главы выделено одиннадцать сущностей: пользователи, команды, приложения, сервисы, тикеты, задачи, подзадачи, комментарии, уведомления, журнал аудита и файловые вложения. Каждой сущности соответствует отдельная таблица, отвечающая за свою предметную область.

Центральной сущностью является задача (tasks): к ней присоединены подзадачи, обращения (через ссылку на задачу), сервис, команда и автор-пользователь. Обращения привязаны к клиенту и приложению, пользователи — к команде, а команда — к своему тимлиду, который тоже является пользователем. Комментарии, уведомления и журнал аудита универсально ссылаются на сущности через пару полей «тип сущности» и «идентификатор сущности». Полная схема данных приведена на рисунке 4.

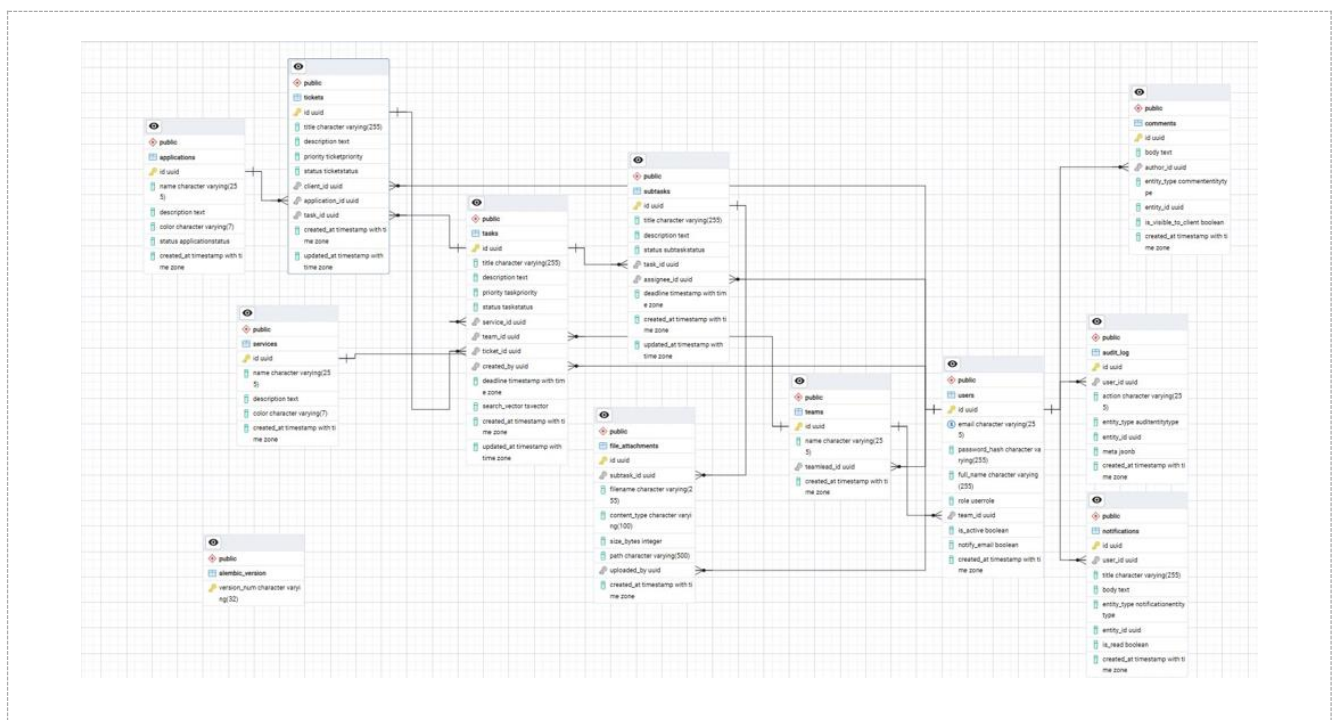


Рисунок 4 — ER-диаграмма базы данных платформы

Из-за большого числа таблиц общая схема довольно плотная, поэтому она приводится также по четырём логическим группам в более крупном масштабе: пользователи и команды (рисунок 5), клиентский модуль (рисунок 6), рабочий модуль (рисунок 7) и системные таблицы (рисунок 8).

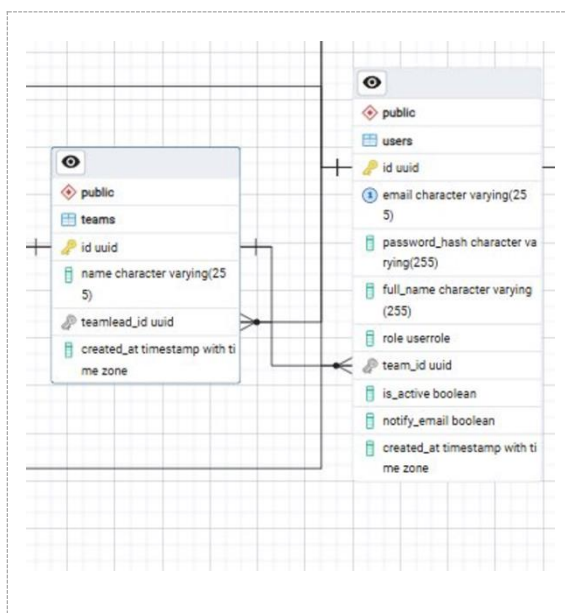


Рисунок 5 — Группа 1: пользователи и команды

Клиентский модуль (рисунок 6) объединяет таблицы, отвечающие за приём и хранение клиентских обращений. Здесь описаны обслуживаемые приложения (продукты) и тикеты, поступающие от внешних заказчиков: каждый тикет связан с приложением, по которому он зарегистрирован, и с породившей его внутренней задачей. Такое разделение позволяет вести учёт обращений отдельно от внутренней работы инженеров второй линии.

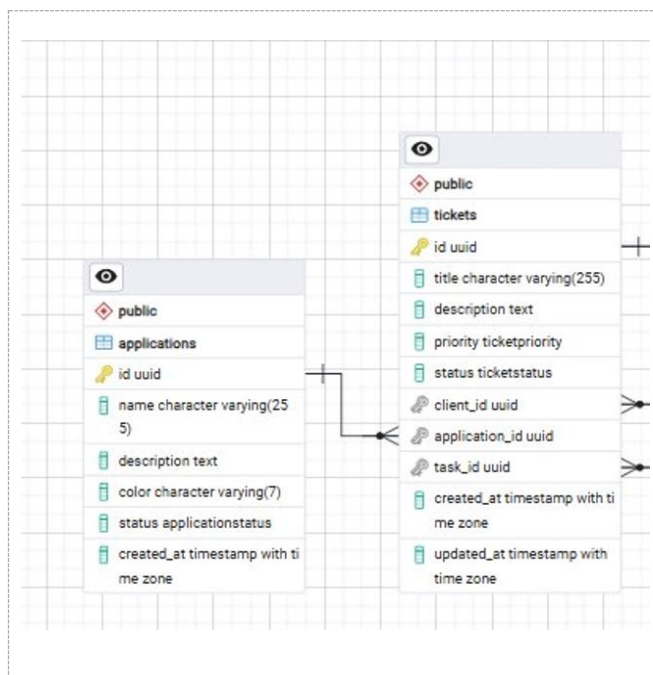


Рисунок 6 — Группа 2: клиентский модуль (приложения, тикеты)

Рабочий модуль (рисунок 7) описывает внутреннюю работу второй линии поддержки. В него входят задачи и подзадачи, по которым ведётся разбор инцидентов, справочник сервисов как направлений работы и вложения к задачам. Задача может содержать несколько подзадач и относиться к определённому сервису, что обеспечивает структурирование и маршрутизацию работы внутри команды.

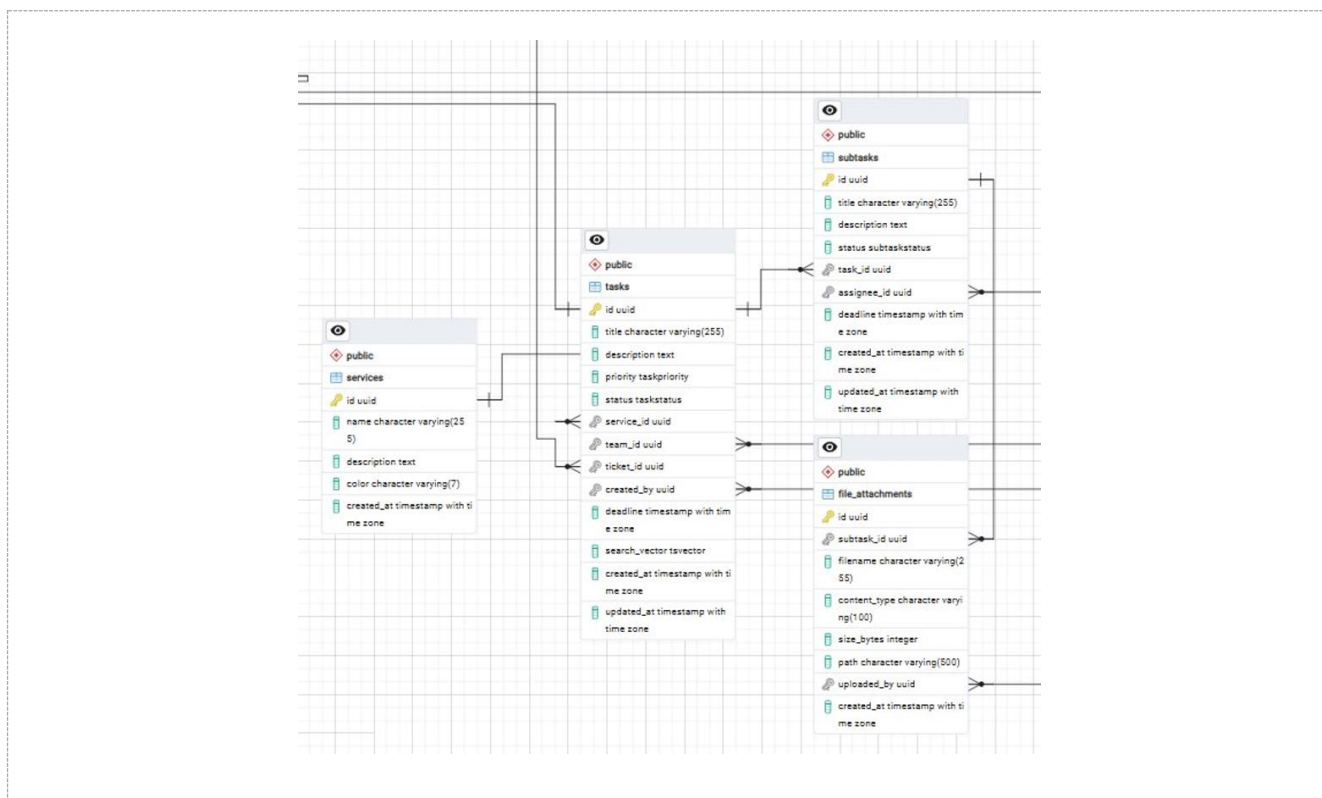


Рисунок 7 — Группа 3: рабочий модуль (задачи, подзадачи, сервисы, вложения)

Системные таблицы (рисунок 8) обеспечивают сопровождающую функциональность платформы. К ним относятся комментарии к задачам и обращениям, уведомления о значимых событиях и записи аудита действий пользователей. Эти сущности связаны с основными таблицами через внешние ключи и поддерживают прозрачность и отслеживаемость происходящего в системе.

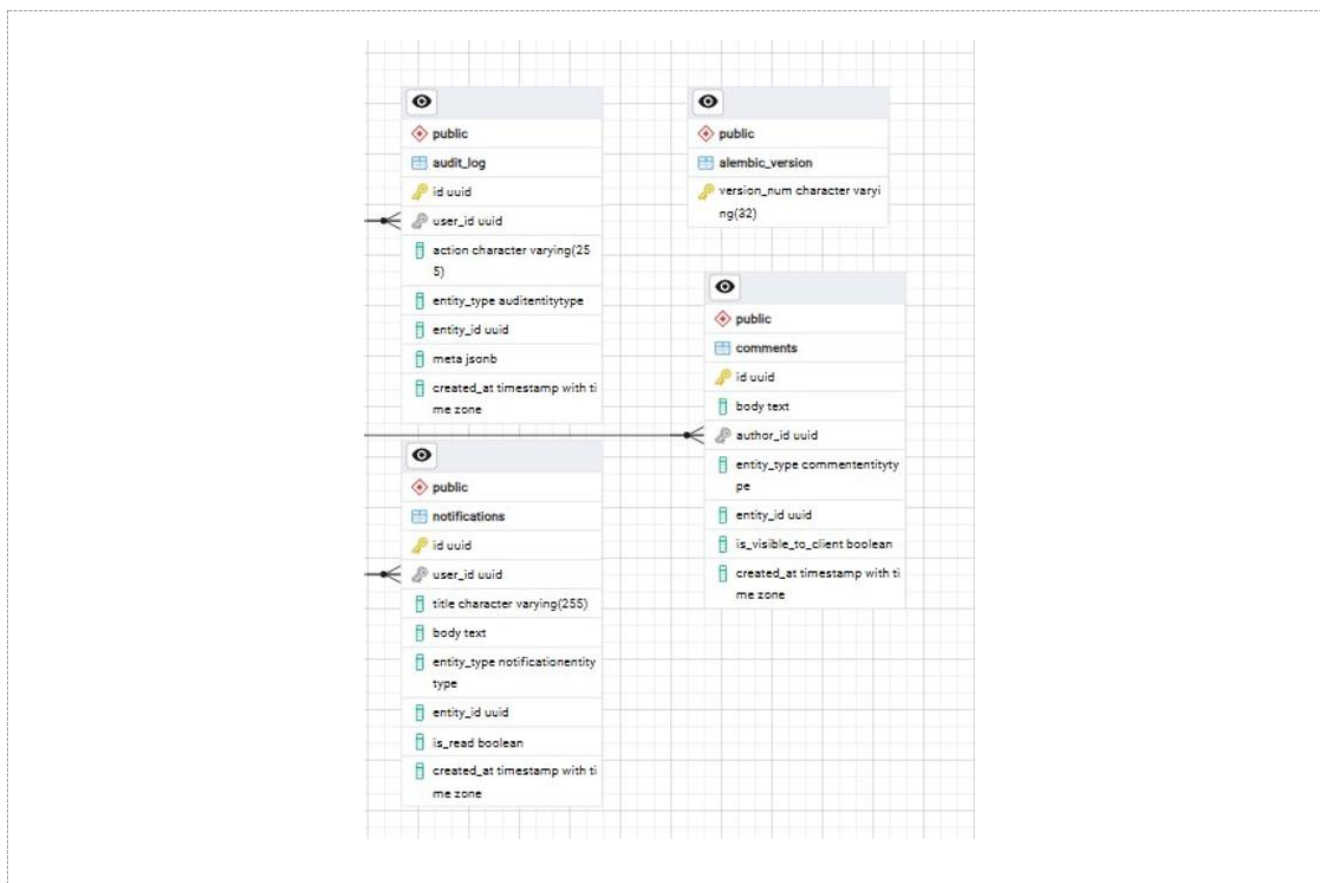


Рисунок 8 — Группа 4: системные таблицы (комментарии, уведомления, аудит)

Ниже подробно описано назначение каждой таблицы, её ключевые поля и связи.

Пользователи (users) — основная сущность. Поля: идентификатор, уникальный адрес электронной почты, хеш пароля, ФИО, роль (одна из пяти), привязка к команде, флаги активности и согласия на почтовые уведомления, дата регистрации. Пароль хранится как bcrypt-хеш с двенадцатью раундами, а адрес электронной почты приводится к нижнему регистру и снабжается уникальным индексом.

Команды (teams) хранят название и ссылку на тимлида, который сам является пользователем. Связь двусторонняя: у пользователя есть привязка к команде, у команды — ссылка на тимлида. Возникающая циклическая ссылка разрешается в миграции порядком создания таблиц.

Приложения (applications) — справочник продуктовых модулей, к которым клиент привязывает обращения: «Личный кабинет», «Админ-панель», «Мобильное приложение», «API Gateway», «Аналитический портал». Поля: название, описание,

цвет для интерфейса, статус (активно или в архиве). Архивный статус нужен, чтобы не удалять записи физически, поскольку они могут быть привязаны к историческим обращениям.

Сервисы (services) — справочник внутренних направлений работ, к которым менеджер привязывает задачи: фронтенд, бэкенд, эксплуатация, мобильная разработка, тестирование, дизайн, аналитика. Они также имеют название, описание и цвет, который служит визуальным маркером для быстрого различения направлений в таблице задач.

Тикеты (tickets) — внешняя сущность, обращение клиента. Поля: название, описание, приоритет, статус (новый, в обработке, принят, отклонён, закрыт), ссылки на клиента и приложение, опциональная ссылка на задачу. Статус автоматически меняется с «нового» на «в обработке», как только менеджер открывает обращение, что снимает ручную работу по его сопровождению.

Задачи (tasks) — центральная единица учёта. Поля: название, описание, приоритет, статус из семи значений (черновик, назначена, в работе, на проверке, выполнена, отклонена, в архиве), привязка к сервису, опциональные привязки к команде и обращению, автор, дедлайн. Жизненный цикл проходит через явные переходы статусов в логике интерфейса. Дополнительно есть автоматически рассчитываемое поле полнотекстового вектора для быстрого поиска.

Подзадачи (subtasks) — декомпозиция задачи. Поля: название, описание, статус (к выполнению, в работе, заблокирована, выполнена), ссылка на задачу с каскадным удалением, ссылка на исполнителя (может отсутствовать) и дедлайн. Тимлид разбивает крупную задачу на подзадачи и распределяет их между инженерами, каждый из которых работает только со своей частью.

Комментарии (comments) — универсальная сущность с полиморфной привязкой через тип и идентификатор сущности (задача, подзадача или обращение). Поля: текст, автор, метка видимости клиенту, дата. Полиморфизм оправдан одинаковой логикой добавления, чтения и удаления для всех трёх типов: три отдельные таблицы потребовали бы дублировать почти идентичные операции.

Уведомления (notifications) хранят по строке на каждое уведомление: получатель, заголовок, текст, тип и идентификатор сущности для перехода по нажатию, флаг прочтения, дата. Индекс по получателю критичен, иначе при росте таблицы выборки превратятся в долгие сканирования; привязка к пользователю выполнена с каскадным удалением.

Журнал аудита (audit_log) фиксирует значимые действия: создание задачи, смену статуса, назначение, удаление пользователя, изменение справочника. Поля: автор действия, строковый код действия, тип и идентификатор затронутой сущности и поле метаданных типа JSONB. Формат JSONB выбран потому, что заранее перечислить все виды метаданных невозможно, а заводить отдельную таблицу под каждый тип события нерационально.

Файловые вложения (file_attachments) привязаны к подзадам и хранят имя файла, его тип, размер, путь на диске и идентификатор загрузившего пользователя. Сами файлы хранятся на примонтированном томе вне базы, поскольку размещение двоичных данных в реляционной СУБД для проекта такого масштаба избыточно и ухудшает резервное копирование.

Модели описаны в синтаксисе SQLAlchemy 2.0 с типизированными колонками. Фрагмент модели пользователя приведён в примере 1.

```
class UserRole(str, enum.Enum):
    client = "client"
    # ... manager, teamlead, worker, admin

class User(Base, UUIDMixin, TimestampMixin):
    email: Mapped[str] = mapped_column(String(255), unique=True,
index=True)
    password_hash: Mapped[str] = mapped_column(String(255),
nullable=False)
    role: Mapped[UserRole] = mapped_column(SAEnum(UserRole))
    # ... full_name, team_id, is_active (см. Приложение 2)
```

Пример 1 — фрагмент модели User (app/models/user.py)

В модели важны три решения. Идентификатор — UUID, а не автоинкремент: он не зависит от состояния базы и безопасен в адресе, не раскрывая число пользователей. Роль — перечисляемый тип, создаваемый в самой базе: это

компактнее и не позволяет записать значение вне списка ролей. Связь с командой — внешний ключ с правилом SET NULL: при удалении команды пользователь не пропадёт, а его привязка обнулится.

Таблица задач устроена сложнее: помимо смысловых полей у неё есть сгенерированное поле полнотекстового вектора, которое на стороне базы постоянно поддерживается в актуальном состоянии. Фрагмент модели приведён в примере 2.

```
class Task(Base, UUIDMixin, TimestampMixin):
    title: Mapped[str] = mapped_column(String(255), nullable=False)
    status: Mapped[TaskStatus] = mapped_column(default=TaskStatus.draft)
    service_id: Mapped[uuid.UUID] =
mapped_column(ForeignKey("services.id"))
    search_vector: Mapped[str | None] = mapped_column(
        TSVECTOR,
        Computed("to_tsvector('russian', title || ' ' || description)")
    # ... priority, team_id, deadline, GIN-индекс (см. Приложение 3)
```

Пример 2 — фрагмент модели Task (app/models/task.py)

Внешний ключ к сервисам с правилом RESTRICT не даёт удалить сервис, к которому привязаны задачи; ключ к командам, наоборот, использует SET NULL — команды могут расформировываться, но задачи при этом не должны пропадать. Дедлайн хранится с учётом часового пояса (везде во всемирном координированном времени), а индекс GIN на поле полнотекстового вектора обеспечивает быстрый поиск.

Начальная миграция написана вручную, чтобы контролировать порядок создания таблиц с циклическими внешними ключами. Циклическая зависимость между пользователями и командами разрешается порядком: сначала создаётся таблица команд без внешнего ключа на тимлида, затем таблица пользователей со ссылкой на команды, и только потом добавляется отложенный ключ. Фрагмент миграции приведён в примере 3.

```
op.create_table("teams",
    sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
    sa.Column("name", sa.String(255), nullable=False),
    sa.Column("teamlead_id", postgresql.UUID(as_uuid=True),
nullable=True))
# users создаётся следом (с FK на teams), и только потом —
# отложенный внешний ключ teams.teamlead_id, разрывающий цикл
```

```
op.create_foreign_key("fk_teams_teamlead_id", "teams", "users",  
    ["teamlead_id"], ["id"], ondelete="SET NULL")
```

Пример 3 — фрагмент миграции (alembic/versions/001_initial.py)

Соединение с базой обеспечивает асинхронный движок с пулом соединений (десять основных и до двадцати дополнительных) и проверкой соединений перед использованием. Значения согласованы с нагрузкой: четыре рабочих процесса по десять соединений дают сорок пиковых, что укладывается в настройки PostgreSQL по умолчанию; при росте нагрузки лимит увеличивается без правки кода. Сессии выдаёт отдельная зависимость, автоматически фиксирующая транзакцию при успешном завершении обработчика и откатывающая её при исключении, так что управление транзакциями берёт на себя инфраструктура, а не каждый обработчик.

Целостность связанных операций обеспечивается транзакциями. Например, при переводе задачи в статус «выполнена» необходимо одновременно закрыть связанное с ней обращение клиента; обе операции выполняются в рамках одной транзакции, поэтому частичное изменение невозможно — либо закрываются и задача, и обращение, либо при ошибке откатывается всё. Перечисляемые типы статусов и приоритетов хранятся в самой базе, что сохраняет самоописательность данных и не позволяет случайно записать недопустимое значение, а тип JSONB в журнале аудита даёт гибкость хранения метаданных событий без отдельной таблицы под каждый тип.

Связь задач и обращений выполнена двусторонней: поле ссылки присутствует с обеих сторон, поэтому из обращения задача создаётся кнопкой с предзаполненной формой, переходы между ними мгновенны, а согласованность полей поддерживают обработчики. При одобрении задачи связанное обращение закрывается автоматически, и клиент получает уведомление — таким образом, одно действие менеджера закрывает и задачу, и обращение. Цветовой атрибут справочников приложений и сервисов проверяется схемой на соответствие формату шестнадцатеричного кода и приводится к единому регистру, что избавляет от расхождений в представлении цвета в интерфейсе и базе.

Для проверки платформы написан скрипт наполнения базы, создающий учётные записи всех ролей, команды, сервисы, приложения, задачи разных статусов, обращения, комментарии и уведомления. Перед запуском скрипт очищает таблицы, поэтому его можно вызывать многократно. Помимо первичных ключей и уникального индекса на адрес электронной почты, заданы индексы на статус, команду и сервис задачи, полнотекстовый вектор, получателя уведомления и связанные поля журнала и вложений; остальные индексы добавляются по факту медленных запросов, а не заранее. В итоге схема покрывает требования первой главы: целостность обеспечивается внешними ключами с осмысленными правилами удаления, самоописательность данных — перечисляемыми типами, а быстрый поиск — индексом GIN.

2.2 Разработка пользовательских интерфейсов

Перед проектированием интерфейсов для каждой роли были расписаны типовые сценарии использования, поскольку одни и те же экраны могут быть удобны одному пользователю и неработоспособны для другого. Анализ сценариев определил состав и компоновку экранов.

Сценарий клиента состоит в создании обращения через форму (тема, описание, приложение, приоритет), отслеживании его статуса в карточке, чтении комментариев менеджера и получении уведомлений в портале и по почте. Отсюда вывод: клиентский интерфейс должен быть спокойным, без таблиц и справочников; ключевой элемент — прозрачная шкала статуса, а сотрудников и структуру компании клиент не видит.

Сценарий менеджера предполагает плотную и быструю работу: фильтр новых обращений, чтение обращения, создание задачи с предзаполнением и автоматической связкой, комментарий клиенту с отметкой видимости, перевод обращения в статус «принят». Поэтому интерфейс менеджера должен быть плотным: описание и связанная задача видны прямо из таблицы, кнопка создания задачи расположена рядом с обращением, а связь обращения и задачи видна с обеих сторон.

Сценарий тимлида строится вокруг канбан-доски команды: уведомление о новой задаче, декомпозиция на подзадачи с исполнителями и дедлайнами, принятие задачи в работу. Главный экран тимлида — доска с перетаскиванием карточек, наглядной загрузкой команды и подсветкой близких дедлайнов. Сценарий инженера сводится к короткому пути «увидеть задачу → сменить статус → прикрепить файл», без аналитики и многоуровневого меню. Сценарий администратора включает создание пользователя (ФИО, адрес, временный пароль, роль, команда), временную деактивацию учётной записи без удаления и просмотр журнала аудита. Главный вывод для интерфейса: у каждой роли свой «первый экран» — менеджеру показатели и обращения, тимлиду доска команды, инженеру «Мои задачи», администратору пользователи, клиенту его обращения.

Интерфейс реализован как два решения на единой технологической базе (React, TypeScript, Vite, Ant Design): внутренний портал — плотный рабочий инструмент, клиентский — минималистичный аккуратный сервис. Дизайн-система — палитра, отступы, радиусы и тени — задана переменными CSS в одном файле, что позволяет перенастроить оформление под брендинг правкой единственного места.

Внутренний портал представляет собой настольное приложение с тёмной боковой панелью слева, на которой расположены поиск, пункты меню (их набор зависит от роли) и блок профиля. Ролевое меню — самое заметное проявление ролевой модели в интерфейсе. Основная область устроена как сочетание верхней панели и содержимого. Главная страница менеджера показана на рисунке 9: четыре плитки-индикатора (всего задач, в работе, просрочено, закрыто за месяц), таблица входящих обращений и таблица активных задач.

Задачи							
4 из 4 задач							
Экспорт CSV Создать задачу Поиск по названию или ID Сервис: все Команда: все Статус: все Приоритет Показано 4							
НАЗВАНИЕ ↓	СЕРВИС	КОМАНДА	ПРИОРИТЕТ	ДЕДЛАЙН	СТАТУС	СОЗДАНА	
☐ Редизайн страницы оформления заказа T-184	Frontend	Frontend	Высокий	22 мая	В работе	2 мая	...
☐ Баг: некорректная сортировка в таблице пользователей T-181	Frontend	Frontend	Средний	18 мая	Назначена	11 мая	...
☐ Оптимизация загрузки главной страницы T-180	Frontend	Frontend	Средний	10 мая	Выполнена	18 апр	...
☐ Рефакторинг модуля авторизации T-176	Frontend	Frontend	Низкий	10 июн	Черновик	13 мая	...

Рисунок 9 — Главная страница (панель показателей) менеджера

Карточки задач и подзадач открываются в выдвижной панели справа, чтобы сохранить контекст списка; внутри размещаются статус, метаданные (сервис, команда, дедлайн), описание, действия и вкладки «Подзадачи», «Комментарии», «История» (рисунок 10).

Переход от старого 3-шагового флоу к single-page с прогрессом. Учесть мобильный сценарий и customer-research отчёт.		
СЕРВИС Frontend	КОМАНДА Frontend	ПРИОРИТЕТ Высокий
ДЕДЛАЙН 22 мая	СОЗДАЛ Алексей Петров	ЗАЯВКА TK-1142
Подзадачи 5 Комментарии 4 История		
2/5 подзадач		
☒ Wireframes основного флоу Анна Иванова до 10 мая Выполнена		
☒ Hi-fi макеты desktop Анна Иванова до 14 мая Выполнена		
☐ Hi-fi макеты mobile Сергей Морозов до 18 мая В работе		
☐ Спект компонента «Stepper» Сергей Морозов до 20 мая Назначена		
☐ Хэндофф разработке Анна Иванова до 22 мая Назначена		
Добавить подзадачу Перевести: В работе Редактировать		

Рисунок 10 — Карточка задачи в выдвижной панели

Главный экран тимлида — канбан-доска из четырёх колонок («Назначена», «В работе», «На проверке», «Выполнено») с перетаскиванием карточек, при

котором смена статуса отправляется на сервер, и переключателем режимов «Доска / Список» (рисунок 11).

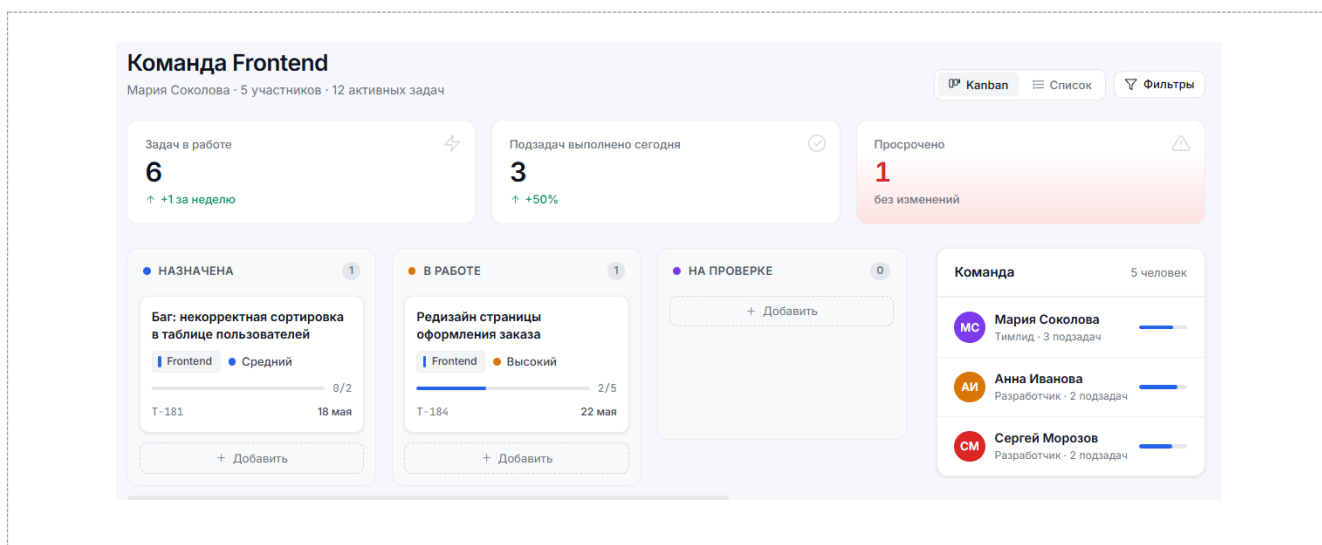


Рисунок 11 — Канбан-доска команды в интерфейсе тимлида

Экран инженера «Мои задачи» содержит карточки подзадач, сгруппированные по родительским задачам, со сменой статуса прямо в карточке; детали, файлы и обсуждение открываются в модальном окне (рисунок 12).

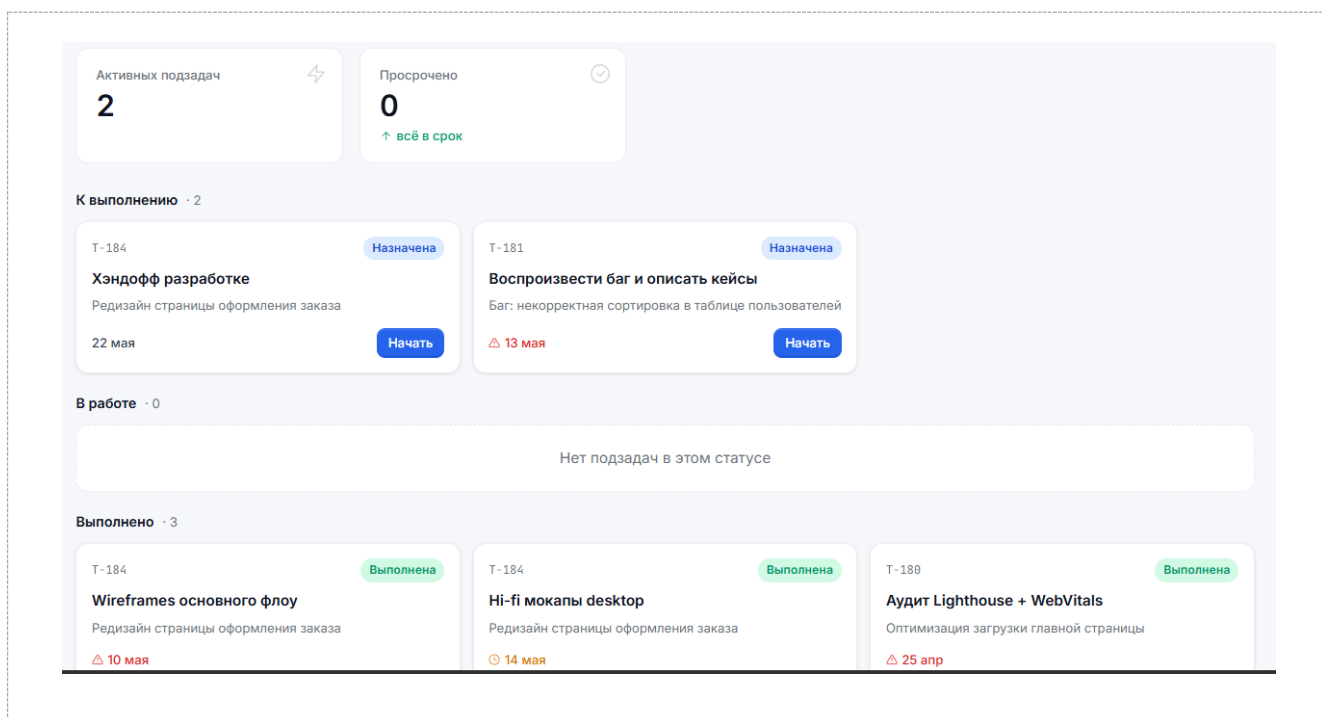


Рисунок 12 — Экран «Мои задачи» исполнителя

Клиентский портал выполнен в светлой гамме с горизонтальным меню (на мобильных устройствах — с нижней панелью вкладок), карточками обращений с цветовой плашкой приложения. Формы входа и регистрации представляют собой одну центральную карточку на спокойном фоне: логотип, поля, кнопка действия и ссылка-альтернатива, без отвлекающих элементов. Ключевой элемент карточки обращения — шкала статусов из четырёх шагов «Новая → В обработке → Принята → Закрыта»: пройденные шаги отмечаются зелёным, текущий — синим, а при отклонении соответствующий шаг подсвечивается красным с подписью об отклонении. Список обращений клиента показан на рисунке 13.

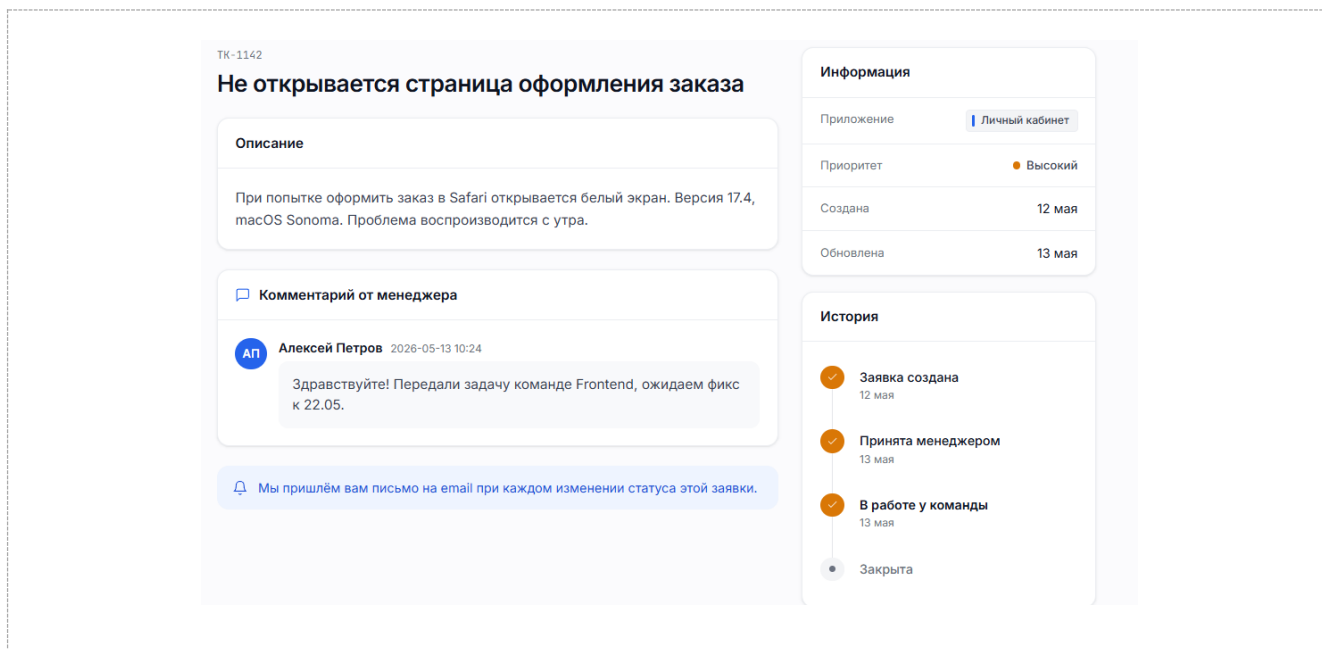


Рисунок 13 — Список обращений в клиентском портале

Подключение клиентов к серверу выполнено через типизированную обёртку над библиотекой `axios` с единым базовым адресом: в эксплуатации запросы идут через веб-сервер, проксирующий их на серверную часть, в разработке — через прокси сборщика. Везде включена передача куки, чтобы токены передавались автоматически. Серверное состояние ведёт `TanStack Query`, клиентское — `Zustand`, при этом источником истины остаётся сервер: результат запроса текущего пользователя при старте перекрывает кэш. Маршрутизация защищена компонентом, принимающим список разрешённых ролей: при несоответствии роли выполняется перенаправление на «свою» главную страницу.

Раздел уведомлений на внутреннем портале представляет собой отдельную страницу, на которую ведёт значок в шапке: список со значком по типу, заголовком, текстом и временем, фильтром «Все / Непрочитанные» и кнопкой «Отметить все прочитанными»; нажатие помечает уведомление прочитанным и ведёт на связанную сущность. На клиентском портале страница устроена аналогично, без типов задач и подзадач. Производительность раздела обеспечивается индексом по получателю, благодаря которому выборки остаются быстрыми, а счётчик непрочитанных рассчитывается одним скалярным запросом; асинхронный процесс сервера удерживает множество одновременных соединений, поэтому для команды в 20–50 человек запас по нагрузке многократный.

Адаптивность обеспечивается стандартными контрольными точками библиотеки Ant Design: клиентский портал полноценно работает на телефоне, тогда как внутренний рассчитан в первую очередь на работу с ноутбука. В обоих порталах в шапке расположен значок уведомлений со счётчиком непрочитанных, а также отдельная страница со списком уведомлений, фильтром «Только непрочитанные» и кнопкой «Прочитать все».

2.3 Реализация серверной логики, системы уведомлений и аналитики

Архитектура серверной части многослойная: как только появляются фоновая обработка, WebSocket и электронная почта, плоская схема «обработчик → база» перестаёт справляться, поэтому компоненты разнесены по слоям заранее. Внутри серверной части выделены три слоя. Слой маршрутов отвечает только за работу с HTTP: разбор параметров, проверку прав, валидацию и отдачу ответа. Слой моделей и схем содержит ORM-сущности и структуры входа и выхода. Слой сервисов содержит сложную логику, которой неудобно в маршрутах: аутентификацию, работу с файлами, отправку уведомлений и менеджер WebSocket. Маршруты разделены по принципу «один модуль — одна сущность».

Структура проекта представляет собой единый репозиторий, в котором серверная часть и оба фроненда разнесены по каталогам.

```
backend/  
  app/
```

main.py	– точка входа FastAPI
config.py	– настройки через переменные окружения
database.py	– движок, сессии, зависимость get_db
dependencies.py	– зависимости (get_current_user и др.)
models/	– модели SQLAlchemy
schemas/	– схемы Pydantic
routers/	– HTTP-маршруты
services/	– сервисный слой
tasks/	– задачи Celery
seed.py	– наполнение базы
alembic/	– миграции
frontend/	
internal-portal/	– внутренний портал (React + TS)
client-portal/	– клиентский портал (React + TS)

Структура проекта (сокращённо)

Точка входа собирает приложение: создаётся объект FastAPI с обработчиком жизненного цикла, подключаются ограничитель частоты запросов, обработчик междоменных запросов, глобальный обработчик исключений и маршрутизаторы. Контейнеризация устроена так, что один и тот же образ используется для серверной части, рабочего процесса и планировщика — меняется только команда запуска, что экономит место. Фронтенды для эксплуатации собираются в статические файлы и отдаются через веб-сервер, в конфигурации которого статика отдаётся с откатом на главную страницу (для корректной работы клиентской маршрутизации), а пути интерфейса и канала уведомлений проксируются на серверную часть.

Управление задачами — смысловой центр платформы. Менеджер создаёт задачу в статусе «черновик», правит и назначает команде — статус становится «назначена», и тимлид получает уведомление. Тимлид принимает задачу в работу, а по завершении отправляет на проверку. Менеджер либо одобряет результат, либо отклоняет с указанием причины, и тогда цикл повторяется. У одобрения есть побочный эффект: связанное с задачей обращение клиента закрывается автоматически, и клиент получает уведомление. Каждый переход реализован отдельным эндпоинтом, а не универсальным обновлением: отдельные обработчики проще читаются, легче тестируются и сами документируют допустимые действия. Фрагмент обработчика отправки задачи на проверку приведён в примере 4.

```
@router.post("/{task_id}/submit-review", response_model=TaskOut)
```

```

async def submit_review(task_id: uuid.UUID,
                        current_user: User = Depends(require_teamlead),
                        db: AsyncSession = Depends(get_db)):
    task = await _get_task_or_404(db, task_id)
    _ensure_status(task, TaskStatus.in progress)    # иначе 409
    task.status = TaskStatus.review
    # ... проверка команды, аудит, уведомления (см. Приложение 4)

```

Пример 4 — фрагмент app/routers/tasks.py

Внутри обработчика три проверки в строгом порядке: задача существует (иначе 404), её статус допускает переход (иначе 409), а текущий пользователь вправе совершить переход для этой задачи (тимлид управляет только задачами своей команды). Остальные переходы устроены так же: назначение требует статуса «черновик» или «отклонена» и существующей команды, принятие — статуса «назначена» и тимлида этой команды, одобрение и отклонение — статуса «на проверке» и менеджера. Список задач поддерживает фильтры по сервису, команде, статусу, приоритету и диапазонам дедлайнов, а также полнотекстовый поиск, реализованный с передачей параметров через привязку, что исключает внедрение кода. Управление справочниками вынесено в два модуля с однотипными операциями: приложения архивируются (не удаляются физически), а сервисы удаляются обычным образом, но правило RESTRICT не даёт удалить сервис, на который ссылаются задачи.

Аутентификация построена на технологии JSON Web Token [19]: после входа сервер выдаёт два токена — доступа на пятнадцать минут и обновления на семь дней, — оба в куках с признаком `httpOnly`, что делает их недоступными из JavaScript и снимает значительный класс атак с кражей токена [18]. Хеширование паролей выполняется библиотекой `passlib` с алгоритмом `bcrypt`, а формирование токенов — библиотекой `python-jose`; все важные параметры вынесены в настройки. Фрагмент создания токена приведён в примере 5.

```

def create_access_token(user_id: uuid.UUID, role: str) -> str:
    now = datetime.now(timezone.utc)
    expire = now +
timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)
    payload = {"sub": str(user_id), "role": role,
               "type": "access", "iat": now, "exp": expire}
    return jwt.encode(payload, settings.JWT_SECRET_KEY,
                      algorithm=settings.JWT_ALGORITHM)

```

Пример 5 — фрагмент app/services/auth.py

Контроль доступа реализован через систему зависимостей FastAPI: функция получения текущего пользователя извлекает его из куки или заголовка и проверяет токен (иначе 401), а построенные поверх неё шорткаты для отдельных ролей пропускают запрос или возвращают 403. Сериализация выполняется через выходные схемы: они не пропускают «лишние» поля, поэтому случайно добавленное внутреннее поле модели в интерфейс не учтёт, пока не объявлено в схеме. Внутренний портал дополнительно обновляет токены автоматически — при ответе 401 перехватчик отправляет запрос обновления и повторяет исходный, ставя параллельные запросы в очередь, чтобы не плодить одновременные обновления. Фрагмент перехватчика приведён в примере 6.

```
api.interceptors.response.use((res) => res, async (error) => {
  const original = error.config;
  if (error.response?.status === 401 && !original._retry) {
    original._retry = true;
    await axios.post("/api/auth/refresh", {}, { withCredentials: true });
    return api(original); // повтор исходного запроса
  }
  // ... очередь параллельных запросов (см. Приложение 5)
  return Promise.reject(error);
});
```

Пример 6 — фрагмент api/client.ts (внутренний портал)

Система уведомлений спроектирована так, чтобы события приходили вовремя, выглядели единообразно и не блокировали ответ интерфейса. Уведомление может появиться у пользователя одним из трёх способов: запись в базе для последующего чтения, мгновенная доставка через WebSocket для подсветки значка уведомлений и письмо на электронную почту для тех, у кого включена соответствующая настройка. Все три канала срабатывают через одну центральную функцию, фрагмент которой приведён в примере 7.

```
async def create_notification(db, user_id, title, body,
  entity_type, entity_id=None) -> Notification:
  notification = Notification(user_id=user_id, title=title, body=body,
  ...)
  db.add(notification); await db.flush()
```

```

    await ws_manager.send_to_user(str(user_id), {...}) # 1) push в
WebSocket
    if (await _get_user(db, user_id)).notify_email:      # 2) email
        send_email_notification.delay(str(user_id), title, body)
    return notification

```

Пример 7 — фрагмент app/services/notification.py

Эта функция вызывается отовсюду, где нужно уведомить пользователя: при назначении задачи команде, при принятии её в работу, при одобрении или отклонении, при создании комментария, при смене статуса обращения и при изменении дедлайнов. Логика «кому и о чём сообщать» описана в соответствующих маршрутах, а механика доставки сосредоточена в одном месте. Для мгновенной доставки написан лёгкий менеджер WebSocket-соединений, хранящий для каждого пользователя список активных соединений (у пользователя может быть открыто несколько вкладок), с методами подключения, отключения и отправки события всем активным сессиям. Перед приёмом соединения соответствующий эндпоинт проверяет токен [11] и сверяет идентификатор пользователя, исключая подписку на чужие уведомления. На стороне клиента написан собственный хук, который подключается при входе, обрабатывает входящие сообщения и автоматически переподключается при разрыве с нарастающей задержкой.

Отправка писем вынесена в фоновую задачу Celery: серверная часть лишь кладёт задачу в очередь, а рабочий процесс разбирает её сам; у задачи предусмотрены повторы при временной недоступности почтового сервера. Планировщик выполняет две регулярные задачи: ежечасную проверку приближающихся дедлайнов (мягкое напоминание авторам и исполнителям задач и подзадач с дедлайном в ближайшие сутки) и ежесуточную архивацию задач, находящихся в статусе «выполнено» более тридцати дней, что держит активные списки чистыми, не теряя историю. Клиент получает уведомления только о своём обращении — смена статуса, видимый ему комментарий, закрытие или отклонение, — тогда как внутренние движения задач его не касаются.

Модуль аналитики отвечает на конкретные вопросы менеджера и тимлида и доступен только тимлиду и выше. Все агрегаты рассчитываются непосредственно

в SQL средствами PostgreSQL, поэтому на уровень приложения попадают уже сводные числа, что делает выборки практически мгновенными. Эндпоинт распределения задач по статусам принимает период и опциональную команду и возвращает пары «статус — количество», которые на клиенте превращаются в горизонтальные индикаторы. Фрагмент расчёта приведён в примере 8.

```
q = (select(Task.status, func.count(Task.id).label("count"))
     .where(and_(*filters))
     .group_by(Task.status))
rows = (await db.execute(q)).all()
data = [StatusCount(status=r.status.value, count=r.count) for r in rows]
```

Пример 8 — фрагмент app/routers/analytics.py

Помимо распределения по статусам, модуль рассчитывает среднее время закрытия (разница между созданием и завершением выполненных задач за период, переведённая в часы; на пустой выборке возвращается признак отсутствия данных), нагрузку команд (число активных задач и подзадач и состав каждой команды, отображаемые столбчатой диаграммой), список просроченных задач (с дедлайном раньше текущего момента, отсортированный так, чтобы самые проблемные были сверху) и статистику по обращениям (общее число за период и разбивка по статусам и приоритетам). Особый интерес представляет эндпоинт недельных срезов: он возвращает параллельный временной ряд — сколько задач создано и сколько закрыто за каждую неделю — за счёт приведения даты к началу недели. Этот срез полезен в динамике: если задач стабильно создаётся больше, чем закрывается, команда копит долг, и это становится видно за недели до возникновения кризиса. Тяжёлая библиотека построения графиков сознательно не подключалась — простые индикаторы и столбчатые диаграммы свёрстаны средствами CSS, что уменьшает число зависимостей и даёт полный контроль над видом.

Конфигурация собрана в единой структуре настроек, читающей переменные окружения: это исключает «магические» константы в коде и при запуске сразу проверяет, что обязательные значения заданы и имеют верные типы, иначе приложение не стартует вовсе. Для типовых операций в корне проекта подготовлен

набор коротких команд: запуск всего стека, применение миграций, наполнение базы демонстрационными данными, просмотр журналов и открытие консоли базы. Это снижает порог входа в проект: достаточно знать несколько команд, не запоминая длинных вызовов. Миграции выполняются штатным инструментом с ручным заполнением операций ради предсказуемости, поскольку автоматическая генерация иногда пропускает изменения перечисляемых типов или фиксирует ложные различия.

Развёртывание и документация. Платформа запускается в Docker одной командой, после чего применяются миграции и при необходимости наполняется база демонстрационными данными; внутренний портал становится доступен на одном порту, клиентский — на другом. Конфигурация задаётся через переменные окружения, описанные в шаблоне, причём секреты не попадают в репозиторий — в него включается только пример. Файл README содержит краткое описание проекта, требования к окружению, инструкцию запуска и список тестовых учётных записей, а для типовых операций подготовлен набор коротких команд. В эксплуатации рекомендуется регулярно создавать резервные копии базы и тома с загруженными файлами; обновление до новой версии сводится к получению свежего кода, пересборке образов, применению новых миграций и перезапуску контейнеров. Согласно методическим указаниям, отдельно задокументировано использование инструментов искусственного интеллекта: они применялись для обсуждения архитектурных вариантов, разбора ошибок при отладке и генерации шаблонного кода (типовых обработчиков, обёрток над клиентом), тогда как архитектура, бизнес-логика переходов статусов, ролевая модель, схема базы данных и интерфейсные решения разработаны самостоятельно, а пользовательские тексты написаны вручную; ссылки на инструменты приведены в файле README.

Заключение

В рамках дипломного проекта спроектирована и реализована веб-платформа для управления рабочими задачами команд второй линии технической поддержки. Полученное решение покрывает сквозной процесс: от создания клиентом обращения до его закрытия через внутреннюю задачу с декомпозицией на подзадачи, согласованием результата и информированием клиента.

В ходе работы решены все поставленные во введении задачи. Проанализирована предметная область и существующие программные решения, выявлены пробелы рыночных систем и сформированы требования к платформе. Построена система из пяти ролей с матрицей прав, действующей на стороне клиента и сервера. Спроектирована и реализована схема базы данных из одиннадцати таблиц со связями и индексами. Обоснован и применён стек разработки: FastAPI, PostgreSQL, React с TypeScript, Celery, Redis и WebSocket для доставки уведомлений.

Определены и применены требования к безопасности: хеширование паролей алгоритмом bcrypt, токены доступа с коротким временем жизни и хранением в защищённых куках, двухуровневая проверка прав на стороне сервера, ограничение частоты запросов, валидация входных данных, контроль типов и размеров загружаемых файлов, а также журнал аудита для расследования инцидентов. Реализована серверная часть с подключёнными интерфейсами: два клиентских приложения на единой базе различаются тональностью и сценариями использования — внутреннее рассчитано на плотную работу сотрудников, клиентское — на создание обращения и отслеживание его статуса.

Подготовлена документация и инструкция по развёртыванию. Платформа запускается в Docker одной командой, миграции применяются штатным инструментом, а скрипт наполнения базы формирует репрезентативный набор демонстрационных данных. Использование инструментов искусственного интеллекта задокументировано отдельно. Из наблюдений по итогам разработки отмечается, что разделение задач и подзадач, а также разделение приложений и

сервисов окупилась, дав два независимых уровня управления и две независимые оси аналитики, а единая функция доставки уведомлений сделала систему заметно отзывчивее.

Платформа реализована как минимально жизнеспособный продукт, покрывающий ключевые сценарии. Естественными направлениями дальнейшего развития являются интеграция с мессенджерами, импорт обращений из внешних систем поддержки по программному интерфейсу, конструктор отчётов в модуле аналитики и поддержка соглашений об уровне обслуживания с автоматической эскалацией просроченных задач. Слоистая архитектура и заложенная расширяемость позволяют добавлять эти возможности без переписывания основы. Результаты работы и практическая часть в виде открытого репозитория доступны для проверки; решение готово к использованию в условиях небольшой и средней команды поддержки и при необходимости может быть адаптировано под нужды конкретной организации.

Список Литературы

1. Бэнкер К. MongoDB в действии / К. Бэнкер ; пер. с англ. А. А. Слинкина. — Москва : ДМК Пресс, 2014. — 394 с.
2. Документация Alembic [Электронный ресурс]. — Режим доступа: <https://alembic.sqlalchemy.org/en/latest/> (дата обращения: 14.04.2026).
3. Документация FastAPI [Электронный ресурс]. — Режим доступа: <https://fastapi.tiangolo.com/> (дата обращения: 14.04.2026).
4. Документация PostgreSQL 15. Full Text Search [Электронный ресурс]. — Режим доступа: <https://www.postgresql.org/docs/15/textsearch.html> (дата обращения: 14.04.2026).
5. Документация SQLAlchemy 2.0 [Электронный ресурс]. — Режим доступа: <https://docs.sqlalchemy.org/en/20/> (дата обращения: 14.04.2026).
6. Дронов В. А. React, Sass, Material-UI. Готовые решения для создания привлекательных адаптивных сайтов / В. А. Дронов. — Санкт-Петербург : БХВ-Петербург, 2022. — 384 с.
7. Конноли Т. Базы данных. Проектирование, реализация и сопровождение. Теория и практика / Т. Конноли, К. Бегг ; пер. с англ. — 3-е изд. — Москва : Вильямс, 2017. — 1440 с.
8. Кочерженко Е. В. Особенности построения систем технической поддержки пользователей / Е. В. Кочерженко // Вестник Воронежского государственного университета. Серия: Системный анализ и информационные технологии. — 2022. — № 3. — С. 142–151.
9. Лутц М. Изучаем Python : в 2 т. / М. Лутц ; пер. с англ. — 5-е изд. — Москва : Вильямс, 2019. — Т. 1: 832 с. ; Т. 2: 720 с.
10. Никсон Р. Создаём динамические веб-сайты с помощью PHP, MySQL, JavaScript, CSS и HTML5 / Р. Никсон ; пер. с англ. — 5-е изд. — Санкт-Петербург : Питер, 2019. — 816 с.
11. Описание протокола WebSocket: RFC 6455 [Электронный ресурс] / Internet Engineering Task Force. — Режим доступа: <https://datatracker.ietf.org/doc/html/rfc6455> (дата обращения: 14.04.2026).

- 12.Официальная документация Celery [Электронный ресурс]. — Режим доступа: <https://docs.celeryq.dev/en/stable/> (дата обращения:).
- 13.Официальная документация React [Электронный ресурс]. — Режим доступа: <https://react.dev/> (дата обращения:).
- 14.Официальная документация TanStack Query [Электронный ресурс]. — Режим доступа: <https://tanstack.com/query/latest> (дата обращения:).
- 15.Официальный сайт Ant Design [Электронный ресурс]. — Режим доступа: <https://ant.design/docs/react/introduce> (дата обращения:).
- 16.Портал 2ЛТП — Дизайн : дизайн-макеты пользовательских интерфейсов системы [Электронный ресурс] / Figma. — Режим доступа: <https://www.figma.com/design/HARy0qPnZwwx5q8SgF5AGb/%D0%9F%D0%BE%D1%80%D1%82%D0%B0%D0%BB-2%D0%9B%D0%A2%D0%9F-%E2%80%94%D0%94%D0%B8%D0%B7%D0%B0%D0%B9%D0%BD?node-id=0-1&t=vUtoUGQzVvqoU11j-1> (дата обращения: 19.06.2026).
- 17.Рамальо Л. Python: к вершинам мастерства / Л. Рамальо ; пер. с англ. — Москва : ДМК Пресс, 2021. — 768 с.
- 18.Скляр Д. Изучаем HTTP. Полное руководство / Д. Скляр ; пер. с англ. — Москва : Эксмо, 2020. — 528 с.
- 19.Сравнение методов аутентификации в современных веб-приложениях [Электронный ресурс] / А. С. Петров // Habr. — Режим доступа: <https://habr.com/ru/articles/793094/> (дата обращения:).
- 20.Стандарт JWT: RFC 7519 [Электронный ресурс] / Internet Engineering Task Force. — Режим доступа: <https://datatracker.ietf.org/doc/html/rfc7519> (дата обращения:).
- 21.Фаулер М. Шаблоны корпоративных приложений / М. Фаулер ; пер. с англ. — Москва : Вильямс, 2018. — 544 с.

Приложение 1

Ссылка на репозиторий проекта

Полный исходный код дипломного проекта, включающий серверную часть на FastAPI, оба фронтенда (внутренний и клиентский порталы), миграции базы данных, скрипт наполнения базы и конфигурацию Docker, размещён в открытом репозитории на платформе GitHub и доступен публично для проверки государственной экзаменационной комиссии.



Ссылка для ручного ввода: <https://github.com/Bibuk/2ltp>

Для перехода в репозиторий с мобильного устройства достаточно отсканировать приведённый выше QR-код стандартной камерой смартфона. Для просмотра с компьютера адрес можно ввести вручную в адресной строке браузера. В корневой директории репозитория размещён файл README.md с описанием проекта, перечнем требуемых для запуска инструментов, пошаговой инструкцией по сборке и развёртыванию, а также списком тестовых учётных записей.

Приложение 2

Модель пользователя User (app/models/user.py)

Полная версия модели User, фрагмент которой приведён в примере 1.

```
class UserRole(str, enum.Enum):
    client = "client"
    manager = "manager"
    teamlead = "teamlead"
    worker = "worker"
    admin = "admin"

class User(Base, UUIDMixin, TimestampMixin):
    __tablename__ = "users"
    email: Mapped[str] = mapped_column(
        String(255), unique=True, nullable=False, index=True)
    password_hash: Mapped[str] = mapped_column(String(255),
nullable=False)
    full_name: Mapped[str] = mapped_column(String(255), nullable=False)
    role: Mapped[UserRole] = mapped_column(
        SAEnum(UserRole, name="userrole", create_type=True),
        nullable=False, default=UserRole.client)
    team_id: Mapped[Optional[uuid.UUID]] = mapped_column(
        UUID(as_uuid=True),
        ForeignKey("teams.id", ondelete="SET NULL"), nullable=True)
    is_active: Mapped[bool] = mapped_column(Boolean, default=True)
    notify_email: Mapped[bool] = mapped_column(Boolean, default=True)
```

Приложение 3

Модель задачи Task (app/models/task.py)

Полная версия модели Task, фрагмент которой приведён в примере 2.

```
class TaskStatus(str, enum.Enum):
    draft = "draft"
    assigned = "assigned"
    in_progress = "in_progress"
    review = "review"
    done = "done"
    rejected = "rejected"
    archived = "archived"

class Task(Base, UUIDMixin, TimestampMixin):
    __tablename__ = "tasks"
    title: Mapped[str] = mapped_column(String(255), nullable=False)
    description: Mapped[str] = mapped_column(Text, default="")
    priority: Mapped[TaskPriority] = mapped_column(
        SAEnum(TaskPriority, name="taskpriority"),
        default=TaskPriority.medium)
    status: Mapped[TaskStatus] = mapped_column(
        SAEnum(TaskStatus, name="taskstatus"), default=TaskStatus.draft)
    service_id: Mapped[uuid.UUID] = mapped_column(
        UUID(as_uuid=True),
        ForeignKey("services.id", ondelete="RESTRICT"), nullable=False)
    team_id: Mapped[Optional[uuid.UUID]] = mapped_column(
        UUID(as_uuid=True),
        ForeignKey("teams.id", ondelete="SET NULL"), nullable=True)
    deadline: Mapped[Optional[datetime]] = mapped_column(
        DateTime(timezone=True), nullable=True)
    search_vector: Mapped[Optional[str]] = mapped_column(
        TSVECTOR,
        Computed("to_tsvector('russian', coalesce(title, '')) || ' ' || "
                "coalesce(description, '')", persisted=True),
        nullable=True)
    __table_args__ = (
        Index("ix_tasks_search_vector", "search_vector",
              postgresql_using="gin"),
        Index("ix_tasks_status", "status"),
        Index("ix_tasks_team_id", "team_id"))
```

Приложение 4

Обработчик `submit_review` (`app/routers/tasks.py`)

Полная версия обработчика, фрагмент которого приведён в примере 4.

```
@router.post("/{task_id}/submit-review", response_model=TaskOut)
async def submit_review(
    task_id: uuid.UUID,
    current_user: User = Depends(require_teamlead),
    db: AsyncSession = Depends(get_db)):
    result = await db.execute(select(Task).where(Task.id == task_id))
    task = result.scalar_one_or_none()
    if not task:
        raise HTTPException(404, "Task not found")
    if task.status != TaskStatus.in_progress:
        raise HTTPException(
            409, "Task must be in_progress to submit for review")
    if current_user.role == UserRole.teamlead \
        and task.team_id != current_user.team_id:
        raise HTTPException(403, "Not your team's task")
    task.status = TaskStatus.review
    task.updated_at = datetime.now(timezone.utc)
    await db.flush()
    await _log_audit(db, current_user.id, "task.submitted_review",
task.id)
    await _notify_managers(db, ...) # уведомляем менеджеров
    return TaskOut.model_validate(task)
```


Приложение 5

Перехватчик обновления токена (api/client.ts)

Полная версия перехватчика axios, фрагмент которого приведён в примере 6.

```
api.interceptors.response.use(
  (res) => res,
  async (error) => {
    const original = error.config;
    if (error.response?.status === 401 && !original._retry) {
      if (isRefreshing) {
        return new Promise((res, rej) =>
          failedQueue.push({ resolve: res, reject: rej })
        ).then(() => api(original));
      }
      original._retry = true;
      isRefreshing = true;
      try {
        await axios.post("/api/auth/refresh", {},
          { withCredentials: true });
        processQueue(null);
        return api(original);
      } catch (e) {
        processQueue(e);
        window.location.href = "/login";
        return Promise.reject(e);
      } finally {
        isRefreshing = false;
      }
    }
    return Promise.reject(error);
  });
```

Приложение 6

Сервис уведомлений `create_notification` (app/services/notification.py)

Полная версия функции `create_notification`, фрагмент которой приведён в примере 7.

```
async def create_notification(
    db: AsyncSession,
    user_id: uuid.UUID,
    title: str,
    body: str,
    entity_type: NotificationEntityType,
    entity_id: Optional[uuid.UUID] = None) -> Notification:
    notification = Notification(
        user_id=user_id, title=title, body=body,
        entity_type=entity_type, entity_id=entity_id, is_read=False)
    db.add(notification)
    await db.flush()
    # 1) push в WebSocket
    await ws_manager.send_to_user(
        str(user_id),
        {"type": "notification", "id": str(notification.id),
         "title": title, "body": body, "is_read": False})
    # 2) email — если пользователь подписан
    user = (await db.execute(
        select(User).where(User.id == user_id))).scalar_one_or_none()
    if user and user.notify_email:
        send_email_notification.delay(str(user_id), title, body)
    return notification
```

Приложение 7

Состав сервисов и команды развёртывания

Перечень контейнеров проекта, описанных в файле `docker-compose.yml`, и основные команды для работы с системой.

```
# docker-compose.yml — состав сервисов
services:
  postgres:      # СУБД PostgreSQL 15, том persistence
  redis:         # брокер очереди Celery и хранилище лимитов
  backend:       # FastAPI + Uvicorn (REST API и WebSocket)
  celery-worker: # обработка фоновых задач (письма)
  celery-beat:   # планировщик периодических задач
  frontend-internal: # внутренний портал (React + nginx)
  frontend-client:  # клиентский портал (React + nginx)

# Основные команды (Makefile)
make up          # запустить весь стек
make migrate     # применить миграции базы данных
make seed       # наполнить базу демонстрационными данными
make logs       # просмотр логов сервисов
make shell-db   # открыть консоль psql

# Развёртывание с нуля
git clone https://github.com/Bibuk/2ltp.git
cp .env.example .env          # задать POSTGRES_PASSWORD и SECRET_KEY
docker compose up -d --build
docker compose exec backend alembic upgrade head
docker compose exec backend python -m app.seed
```

После запуска внутренний портал доступен по адресу `http://localhost:3000`, клиентский — по `http://localhost:3001`. Тестовые учётные записи (`admin@2ltp.ru`, `manager@2ltp.ru`, `tl.frontend@2ltp.ru`, `worker1@2ltp.ru`, `client@2ltp.ru` и другие) с паролями вида «Admin123!» печатает скрипт наполнения базы; этого набора достаточно для проверки сценариев всех ролей.

Приложение 8

Использование ИИ-инструментов в проекте

В соответствии с методическими указаниями в данном приложении задокументированы все ИИ-инструменты, использованные при выполнении дипломного проекта, с указанием их назначения и вклада в результат. Искусственный интеллект применялся исключительно для вспомогательных задач, тогда как ключевые этапы работы — концепция, проектирование, написание серверного и клиентского кода, анализ и формулирование выводов — выполнены студентом лично. Все материалы, полученные с помощью ИИ, проверены студентом на корректность и достоверность. Перечень использованных инструментов приведён в таблице 2.

Инструмент	Назначение в проекте	Официальный ресурс
Claude (Anthropic)	Вспомогательная помощь при проработке дизайна пользовательских интерфейсов (варианты компоновки и визуальной иерархии). Сам дизайн и его реализация в коде выполнены студентом.	https://claude.ai
ChatGPT (OpenAI)	Генерация тестовых (демонстрационных) данных для наполнения базы данных — рутинная задача, прямо разрешённая методическими указаниями.	https://chatgpt.com

Таблица 2 — ИИ-инструменты, использованные в проекте

Claude (помощь с дизайном). Инструмент применялся как вспомогательное средство при проработке визуального оформления пользовательских интерфейсов: рассматривались варианты компоновки экранов, организации навигации и визуальной иерархии элементов. Итоговые дизайнерские решения принимались студентом, а их реализация в коде клиентских приложений на React выполнена студентом самостоятельно. Готовый исходный код интерфейсов средствами ИИ не генерировался.

ChatGPT (генерация тестовых данных). Инструмент применялся для автоматизации рутинной задачи — формирования набора демонстрационных (тестовых) данных, которыми наполняется база данных при первом запуске:

учётных записей пользователей разных ролей, перечня приложений и сервисов, примеров обращений, задач и подзадач. Такое использование прямо отнесено методическими указаниями к допустимым (автоматизация рутинных задач, в том числе генерация тестовых данных). Сгенерированные данные проверены студентом на соответствие модели данных и ограничениям целостности, после чего интегрированы в скрипт наполнения базы (модуль `app.seed`, см. Приложение 7). Данные являются вымышленными и не содержат реальных персональных сведений.

Ниже приведён иллюстративный фрагмент сгенерированных тестовых данных (полный набор формируется скриптом наполнения базы):

```
# демонстрационные пользователи (app/seed) – фрагмент
manager@2ltp.ru      | Менеджер Тестовый      | manager
tl.frontend@2ltp.ru  | Тимлид Фронтенд       | teamlead
worker1@2ltp.ru     | Инженер Первый        | worker
client@2ltp.ru      | Клиент Демонстрационный | client
```

Аналогичный перечень ИИ-инструментов с указанием их назначения и ссылок на официальные ресурсы приведён в файле `README.md` репозитория проекта. Ответственность за корректность, точность и качество всех материалов, полученных с применением ИИ, несёт студент.

Приложение 9

Ссылка на дизайн-макеты проекта в Figma

Дизайн-макеты пользовательских интерфейсов дипломного проекта «Портал 2ЛТП» — внутреннего и клиентского порталов — выполнены в сервисе Figma и размещены в открытом доступе для просмотра. Файл содержит экраны для всех ролей системы и отражает компоновку, навигацию и визуальное оформление интерфейсов, реализованных впоследствии в коде на React.



Ссылка для ручного перехода:

<https://www.figma.com/design/HARy0qPnZwwx5q8SgF5AGb/%D0%9F%D0%BE%D1%80%D1%82%D0%B0%D0%BB-2%D0%9B%D0%A2%D0%9F-%E2%80%94%D0%94%D0%B8%D0%B7%D0%B0%D0%B9%D0%BD?node-id=0-1&t=vUtoUGQzVvqoU11j-1>

Для просмотра достаточно перейти по приведённой ссылке в браузере; вход в учётную запись Figma не требуется. Полный исходный код пользовательских интерфейсов, реализующий приведённый дизайн, размещён в репозитории проекта (см. Приложение 1).